

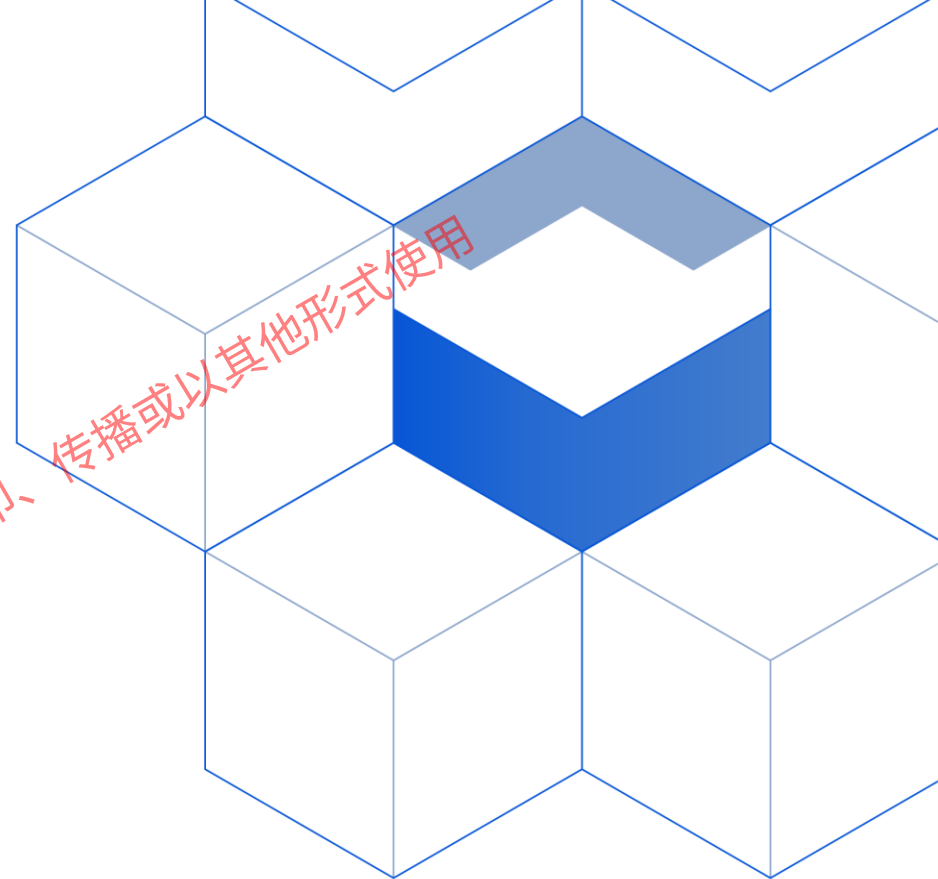
Syslab基础

汇报人：王春雨

日期：2025年7月11日

@苏州同元软控信息技术有限公司版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用



CONTENTS

目录 ➔



用户界面与设置



常见命令与基础知识



脚本构建与代码调试



数据导入导出及可视化



案例实现-太阳能电池板性能验证

未经授权，不得复制或传播或以其他方式使用
@苏州同元软控信息技术有限公司版权所有，侵权必究

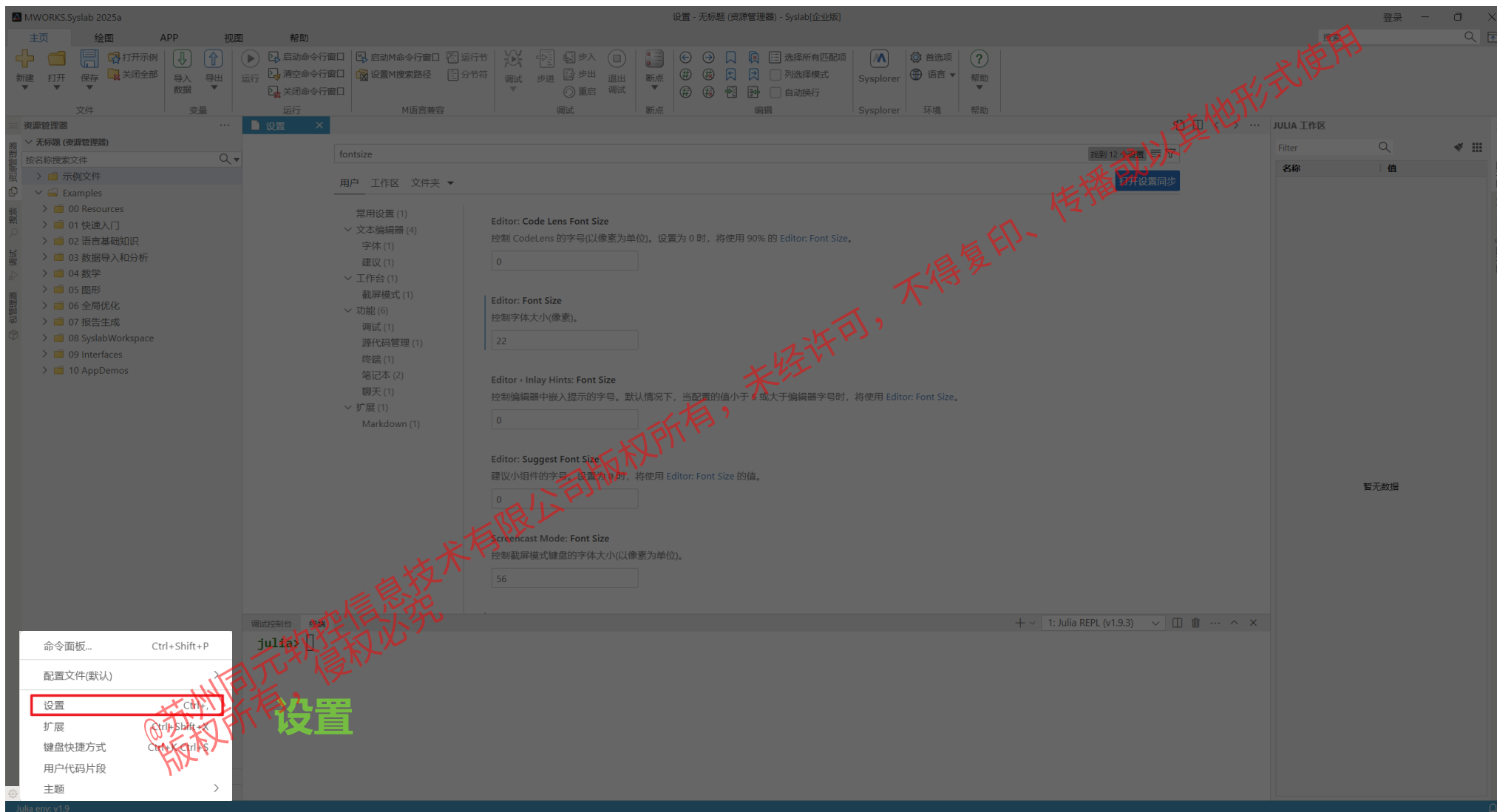
01

PART 01 ➔

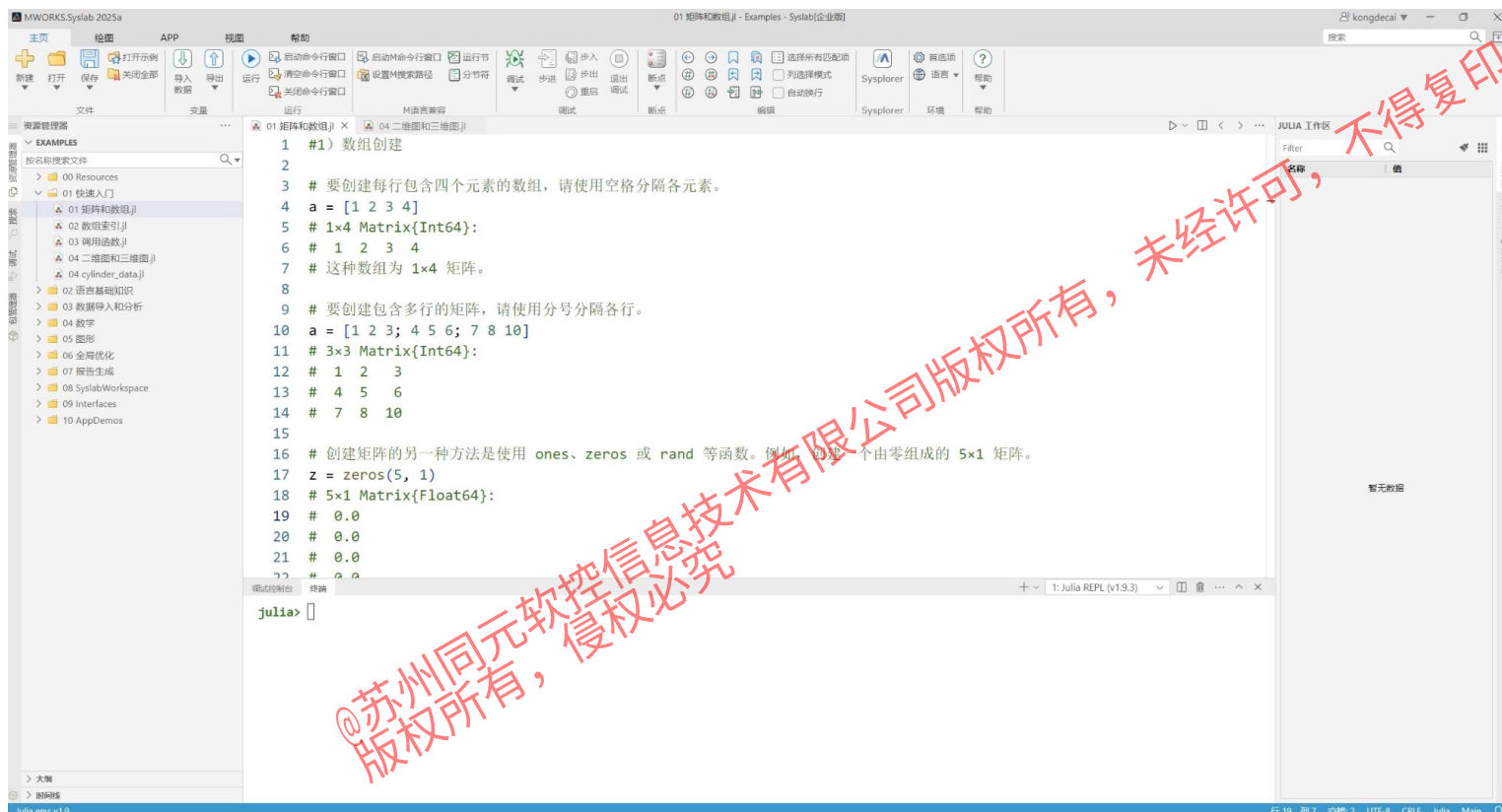
用户界面与设置

©苏州同元软件技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1.1 用户界面



1.1 用户界面

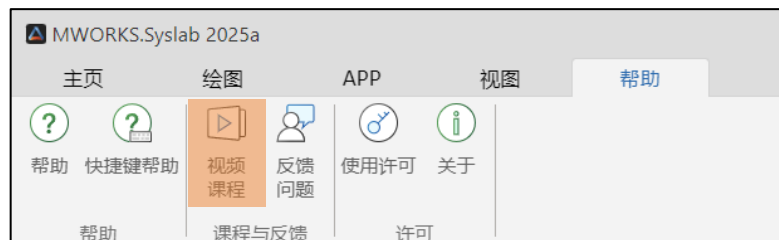


- 1 个基础环境
- 5 个行业领域
- 4100+ 函数帮助
- 支持Julia和M函数双向搜索
- 优化帮助文档结构
(使用Syslab、工具箱、工作流)
- 支持分类主题
(文档、示例、函数、App)
- 支持全局搜索

Syslab帮助中心.mp4

1.1 用户界面

视频课程：提供Syslab软件使用视频课程



1.1 用户界面

MoHub：设置技术交流/高校专区，提供大量常见问题FAQ，安排专业工程师解答

Q

MoHub

搜索

MoHub首页：www.mohub.net

MoHub

首页

技术交流

众创共享

数字教学

创新实践

高校专区

搜索您所感兴趣的...

📖

🔔

👤

专栏

Syslab基础平台

Julia语言

工具箱

Sysplorer基础平台

AI for MWORKS

Modelica语言

模型库

NUMAP

其他

标签

安装激活

系统建模

科学计算

AI

控制工程

工业机器人

船舶

汽车

核能

航空航天

展开更多

高校专区

技术交流专区

问题搜索

最新

推荐

热门

官方配套课程资料现已正式上线!

尊敬的各位用户: "官方课程(https://www.tongyuan.cc/education)" (https://www.mohub.net/2024-12-02)

MoHub · 2024-12-02

2

382

0

系统建模

科学计算

方程数大于300应该怎么办?

1

lc001129 · 2025-03-05

0

3

1

系统建模

这个模型库在哪

"微信图片_20250305092915.png" (https://mohub.net/mohub/resource/20250305092915.png)

xzg · 2025-03-05

0

7

1

系统建模

电机模型

请问一下, 新款的电机怎么和这个V D接口连接呀, 以前的电机有两个接口, 现在的只有一个"微信图片_20250305092915.png"

我要发帖

资料中心

官网培训课程配套资料

模型库手册及配套案例

智能装备系统模型集成—2024蓝...

智能装备控制算法开发—2024蓝...

MWORKS课程资料-预览版

智能装备数字样机构建—2024蓝...

MWORKS.Sysplorer用户手册

MWORKS.Syslab用户手册

FAQ

Modelica如何调用Refprop

模型连线时提示"连接器类型不匹...

Sysblock问题合集

"组件的类型"***找不到, 组件全...

使用Fromworkspace时, 出现"Erro...

出现"Error:divide-zero error"问题...

代码出现"DimensionMismatch: di...

Julia代码中变量已经初始化定义, ...

电学模型仿真出现方程奇异错误怎...

客服

🔊

🗨

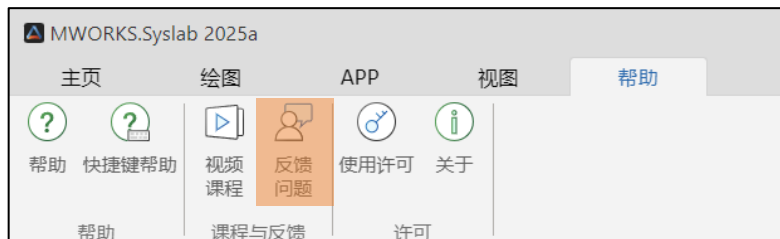
不得打印、传播或以其他形式使用

©苏州同元软控信息技术有限公司版权所有，未经许可，不得转载

苏州大学邀请码：
sudaTy365L

1.1 用户界面

反馈问题：可针对不同产品模块，反馈产品缺陷和提出功能建议等



建议类型

- 产品缺陷
- 功能建议
- 用户体验
- 一般咨询

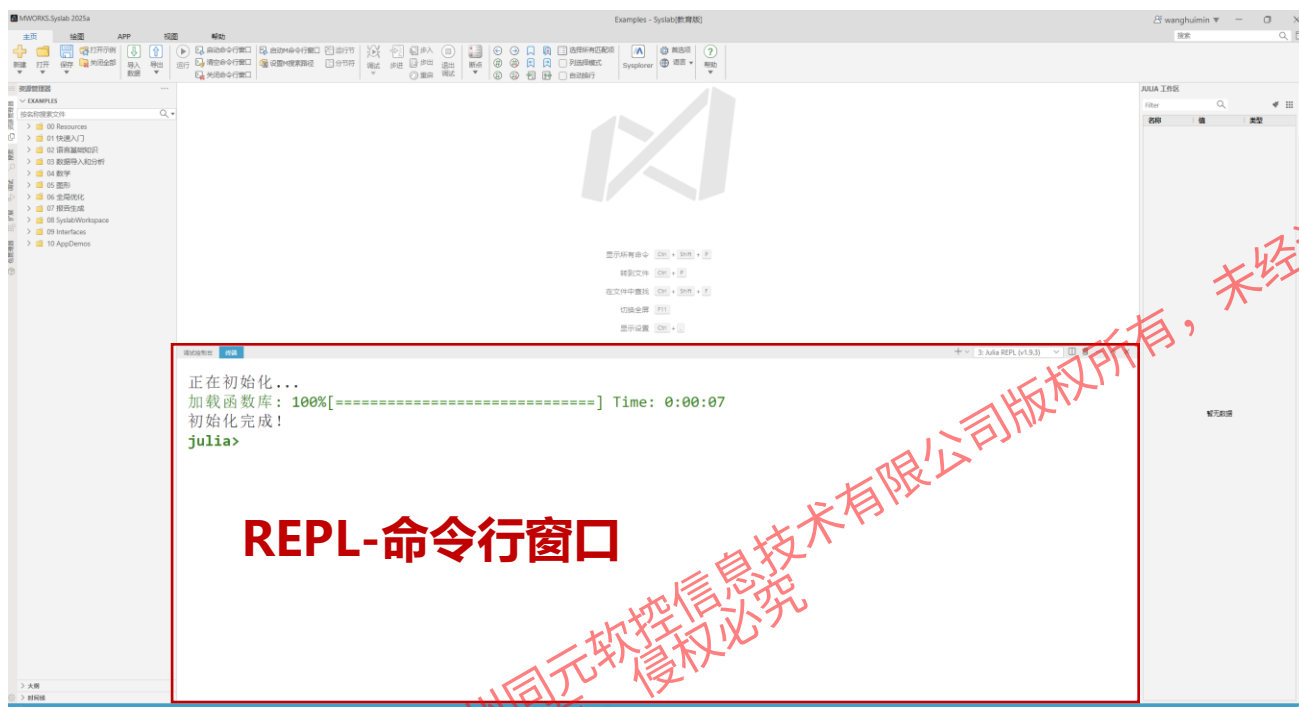
产品模块

- 基本环境
- 外部语言接口
- M语言兼容工具
- 符号数学函数库
- 曲线拟合函数库
- 信号处理函数库
- 通信函数库
- DSP系统函数库
- ...

The screenshot shows the '我想解决的问题' (The problem I want to solve) form. The form includes a '建议类型' (Suggestion Type) dropdown menu with '1. 产品缺陷' (1. Product Defect) selected. Below this is a '产品名称' (Product Name) field with 'MWORKS.Syslab' entered. The '产品模块' (Product Module) dropdown menu is currently empty, showing '请选择模块' (Please select a module). The '问题描述' (Problem Description) field is a large text area with a placeholder '请输入' (Please enter) and a character count of '0/500'. Below the text area are links for '产品信息' (Product Information), '系统信息' (System Information), and an '上传附件' (Upload Attachment) button. At the bottom, there is a section titled '通过以下方式提醒我工单进展' (Remind me of my ticket progress in the following ways) with an email field and a '提交' (Submit) button. A large red watermark is overlaid diagonally across the form, reading '@苏州元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用'.

1.2 基础设置-交互式命令行

Julia REPL 是一个交互式命令行工具，通过 REPL 可以实时输入并执行 Julia 代码。如在其他环境中，可以在终端中输入 Julia 命令进入 Julia REPL，通过快捷键 Ctrl + D 或者调用 exit() 即可退出 REPL。



REPL-命令行窗口

在 REPL 中执行 Julia 程序

进入 REPL 后即可在窗口中输入任何需要执行的 Julia 程序，按下回车键即可在 REPL 的全局作用域执行输入的程序。

在 REPL 中安装包

用户可以通过在默认模式中按下] 键进入包管理模式，通过按下 CTRL+C 或在行首按退格键退出该模式。在该模式中，用户可以通过执行包管理命令加载或安装 Julia 包。

高效的使用 Julia REPL

基于历史回滚输入：

Julia REPL 可以基于已经输入的内容回滚历史输入（快捷键：上方向键），用户可以通过该特性高效的查找历史输入。

使用 TAB 进行自动补全：

在 REPL 中可以输入一个函数或类型的前几个字符，然后按下 Tab 键触发自动补全功能，当 REPL 检测到多种补全项后，将展示所有匹配项的列表

在 REPL 中查找帮助文档

用户可以通过输入问号 ? 进入 REPL 的帮助模式。用户在此模式下输入内容时，Julia 将基于输入内容提供相应的帮助文档。

1.2 基础设置-工具箱加载

点击主页菜单栏中的  首选项，设置**预加载**同元商业工具箱，每次软件启动自动加载函数库



路径
Julia 工作区
变量编辑器
> 文本编辑器
 预加载
 调试
语言服务
导出文件
> Julia 命令行窗口
 Python 命令行窗口
包服务器
帮助
日志
更多配置项

预加载

☒ 控制启动命令行窗口时是否预加载函数库

高级配置

- 基础
 - ☒ 基础库(TyBase)
 - ☒ 数学库(TyMath)
 - ☒ 图形库(TyPlot)
 - ☐ 图像库(TyImages)
 - ☐ 地理图库(TyGeoGraphics)
 - ☐ 报告生成库(TyReportGenerator)
- 数学、统计和优化
 - ☒ 统计库(TyStatistics)
 - ☒ 曲线拟合库(TyCurveFitting)
 - ☐ 符号计算库(TySymbolicMath)
 - ☐ 优化库(TyOptimization)
 - ☐ 全局优化库(TyGlobalOptimization)
- 控制系统
 - ☒ 控制系统库(TyControlSystems)
 - ☐ 系统辨识库(TySystemIdentification)
 - ☐ 鲁棒控制库(TyRobustControl)
- 信号处理和无线通信
 - ☒ 信号处理库(TySignalProcessing)
 - ☒ DSP 系统库(TyDSPSystem)
 - ☒ 通信库(TyCommunication)
 - ☐ 相控阵库(TyPhasedArray)
 - ☐ 雷达库(TyRadar)
 - ☐ 小波库(TyWavelet)

Syslab 2025a 默认加载3个函数库：

- 基础库 (TyBase)
- 数学库 (TyMath)
- 图像库 (TyPlot)

勾选预加载后，需重启命令行窗口：

1. 关闭命令行窗口
2. 启动命令行窗口

1.2 基础设置-工具箱加载

在Julia中通过using使用已正确安装的工具箱，可直接调用工具箱函数

```
using TyMathCore
using MatrixPencils
using MLDatasets: MNIST
using Base.Iterators: partition
using Printf, BSON
using Parameters: @with_kw
using CUDA
```

```
model = sum(A) #直接调用已预加载工具箱中的函数sum对数组A求和
```

```
julia> spzeros
ERROR: UndefVarError: `spzeros` not defined
#当多个库中的函数产生冲突时，需要使用库名+函数名的方式
julia> MatrixPencils.spzeros
spzeros (generic function with 1 method)
```

TyMathCore和MatrixPencils
库中均有spzeros函数

1.2 基础设置-工具箱加载

在Syslab中可以通过包管理器，管理注册库与开发库，安装和卸载工具箱

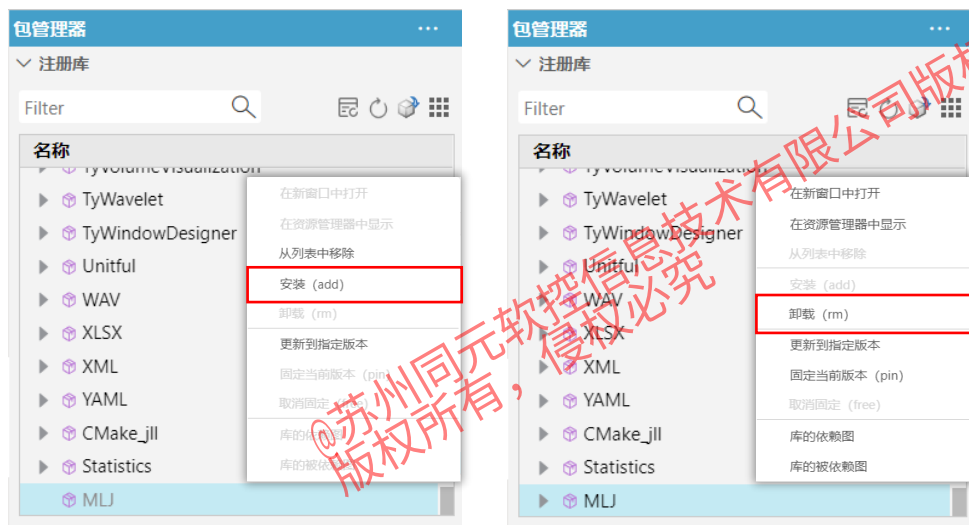
□ 方式1：可视化操作

安装操作步骤：

1. 注册库中点击“添加包”
2. 搜索第三方工具箱，选择对应工具箱，添加至注册库列表
3. 在注册库列表中右击，选择“安装”

卸载操作步骤：

1. 注册库中右击包，选择卸载



安装

卸载

□ 方式2：文本操作

安装操作步骤：

1. 在REPL中输入 `J` 进入pkg模式(退出pkg模式可使用Backspace或 `Ctrl + C`)
2. 在pkg模式中输入： `add 第三方工具箱名`

卸载操作步骤：

1. 在pkg模式中输入： `remove 第三方工具箱名`

```
(@v1.9) pkg> add MLJ
```

```
(@v1.9) pkg> remove MLJ
```

```
(@v1.9) pkg> add MLJ
```

安装

```
(@v1.9) pkg> remove MLJ
```

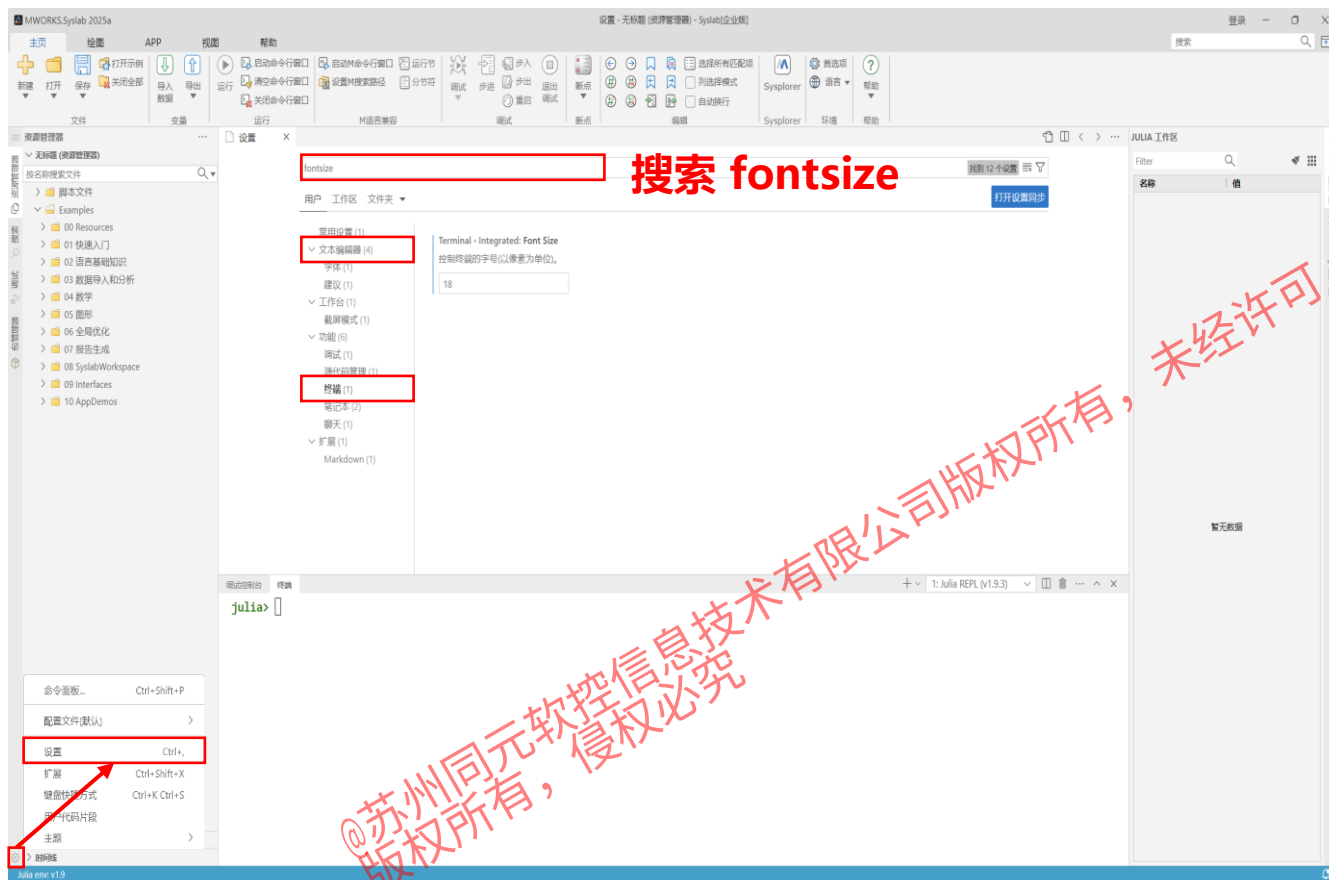
卸载

说明：

- 同元商业工具箱无需安装，可直接使用

1.2 基础设置-字体显示

点击软件左下角设置 ，搜索 fontsize，更改**文本编辑器**或**终端**等字号大小



终端数据格式显示设置：首选项里提供3种设置，分别为自适应(default)、长固定(long)、短固定(short)，可根据需求进行选择



02

PART 02 ➔

常见命令及基础知识

@苏州同元软件技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

2.1.1 常见操作命令

命令	功能	命令	功能
ctrl+L/clc()	清空命令行窗口	save	保存内存变量
clear()	清除工作区变量	load	加载制定文件中的变量
clf()	清除图形窗口	hold("on")	图形保持命令
println	显示变量或文字的内容	cd/pwd/mv	更改当前文件夹/显示当前文件夹/移动或重命名文件或文件夹

使用无参数函数，需要加上对应的括号；

```
julia> a = [1 2 3; 4 5 6; 7 8 10]
3×3 Matrix{Int64}:
 1  2  3
 4  5  6
 7  8 10
```

```
julia> clear()
```

```
julia> a
```

```
for i in 1:5
    println(i)
end
```

```
p = rand(1, 10)
q = ones(10)
filename = "test.mat"
save(filename, "-mat"; p, q)
```

```
julia> load("test.mat")
Dict{String, Any} with 2 entries:
 "q" => [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
 "p" => [0.553362 0.246858 ... 0.0099528 0.0955967]
```

2.1.2 快捷键命令

按钮和快捷键	功能	按钮和快捷键	功能	按钮和快捷键	功能
↑	调用上一行	Ctrl + ←	光标左移一个单词	Alt+backspace	清除当前输入字段
↓	调用下一行	Ctrl + →	光标右移一个单词	Del	删除光标处字符
←	光标左移一个字符	Home	光标置于当前行开头	backspace	删除光标前字符
→	光标右移一个字符	End	光标置于当前行结尾	Ctrl + D	关闭终端

即时查询函数

在 REPL 环境中，不仅仅可以使用代码补全，我们还可以利用代码补全功能即时地查询函数的使用方法

```
julia> sum(
```

```
julia> sum(  
sum(f, x::FillArrays.AbstractFill) @ FillArrays C:\Users\Public\TongYuan\julia\package  
s\FillArrays\KpKaA\src\FillArrays.jl:496  
sum(f::Union{Function, Type}, a::StaticArraysCore.StaticArray{<:Tuple, T}; dims) where  
T @ StaticArrays C:\Users\Public\TongYuan\julia\packages\StaticArrays\MSJcA\src\mapred  
uce.jl:289  
sum(f, a::StaticArraysCore.StaticArray{<:Tuple, T}; dims) where T @ StaticArrays C:\Use  
rs\Public\TongYuan\julia\packages\StaticArrays\MSJcA\src\mapreduce.jl:288  
sum(r::OffsetArrays.OffsetVector{T, <:AbstractRange{T}} where T) @ OffsetArrays C:\User  
s\Public\TongYuan\julia\packages\OffsetArrays\hwmnB\src\OffsetArrays.jl:594  
sum(A::AbstractArray, w::StatsBase.AbstractWeights, dims::Int64) @ StatsBase deprecated  
.jl:103
```

输入 “sum(” 之后按下Tab键，这样就可以看到sum函数以及从它衍生出的各种方法了。回显内容中也包含了每个函数或方法所在的模块和源码文件。

2.1.3 数据输出格式

数据输出格式 `ty_format("style",x)`

Style	结果	示例
default	Julia语言默认数值打印格式	π
short	短固定十进制小数点格式，小数点后包含 4 位数。这是默认数值设置。	3.1416
long	长固定十进制小数点格式，Float64 值的小数点后包含 15 位数，Float32 值的小数点后包含 7 位数。	3.141592653589793
shortE	短科学记数法，小数点后包含 4 位数。	3.1416e+00
longE	长科学记数法，Float64 值的小数点后包含 15 位数，Float32 值的小数点后包含 7 位数。	3.141592653589793e+00
shortG	短固定十进制小数点格式或科学记数法（取更紧凑的一个），总共 4 位。	3.142
longG	长固定十进制小数点格式或科学记数法（取更紧凑的一个），对于 Float64 值，总共 15 位；对于 Float32 值，总共 7 位。	3.141592653589793e+00
shortENG	短工程记数法，小数点后包含 4 位数，指数为 3 的倍数。	3.1416E0
longENG	长工程记数法，包含 15 位有效位数，指数为 3 的倍数。	3.14159265358979E0
+	正/负格式，对正、负和零元素分别显示 +、- 和空白字符。	+
bank	货币格式，小数点后包含 2 位数。	3.14
hex	二进制双精度数字的十六进制表示形式。	400921fb54442d18
rat	小整数的比率。	165707065//52746197

```
julia> pi
π = 3.1415926535897...

julia> ty_format("default",pi)
π

julia> ty_format("short",pi)
3.1416

julia> ty_format("long",pi)
3.141592653589793
```

2.2 基础知识-变量

变量赋值: 与其他动态类型语言一样, Julia 无需提前声明变量类型, 可以直接创建, 提供了灵活的命名策略

```
julia> a = "Hello, World"
"Hello, World"
julia> 啊 = 10 #不推荐使用
10
julia> β = 36 #\beta,再敲击Tab键
36
julia> α^p = 10 #先转义\alpha,在转义\^p
julia> 12a = 12 #不能以数字开头, 12a含义为12*a
```

```
julia> x = [1, 2, 3, 4];
julia> y = x;
julia> y[1] = 0;
julia> x
4-element Vector{Int64}:
 0
 2
 3
 4
julia> y = copy(x);
julia> y[2] = 0;
julia> x
4-element Vector{Int64}:
 0
 2
 3
 4
```

- 不能使用Julia中已有的单词的关键词作为变量名:
`iskeyword()`可以查找已有关键词
- 区分大小写
- 不能以数字开头
- 如果 x 是一个变量, $y = x$ 并不是复制 x 的值给变量 y , 对于可变类型的 x , 对 y 做修改时 x 也会被修改, 如果需要复制 x 的值需要用 $y = \text{copy}(x)$ 或者 $y = \text{deepcopy}(x)$

2.2.1 基础知识-变量

系统预定义的特殊变量

符号	意义	无限精度值	Unicode 字符代码
pi/ π	圆周率	# 3.1415...	\pi
ans	结果显示默认变量名	\	\
exp(1)	自然常数	# 2.7182...	\euler
im	虚数单位	# real(im * im) == -1	

```
pi
# $\pi$  = 3.1415926535897...
eps()
#2.220446049250313e-16

#复数的定义
1+2im
```

在 Julia 中，变量 ans 被设置为交互式会话中提交的最后一个命令的值；MATLAB中ans 被设置为交互式会话中提交的最后一个表达式的值。

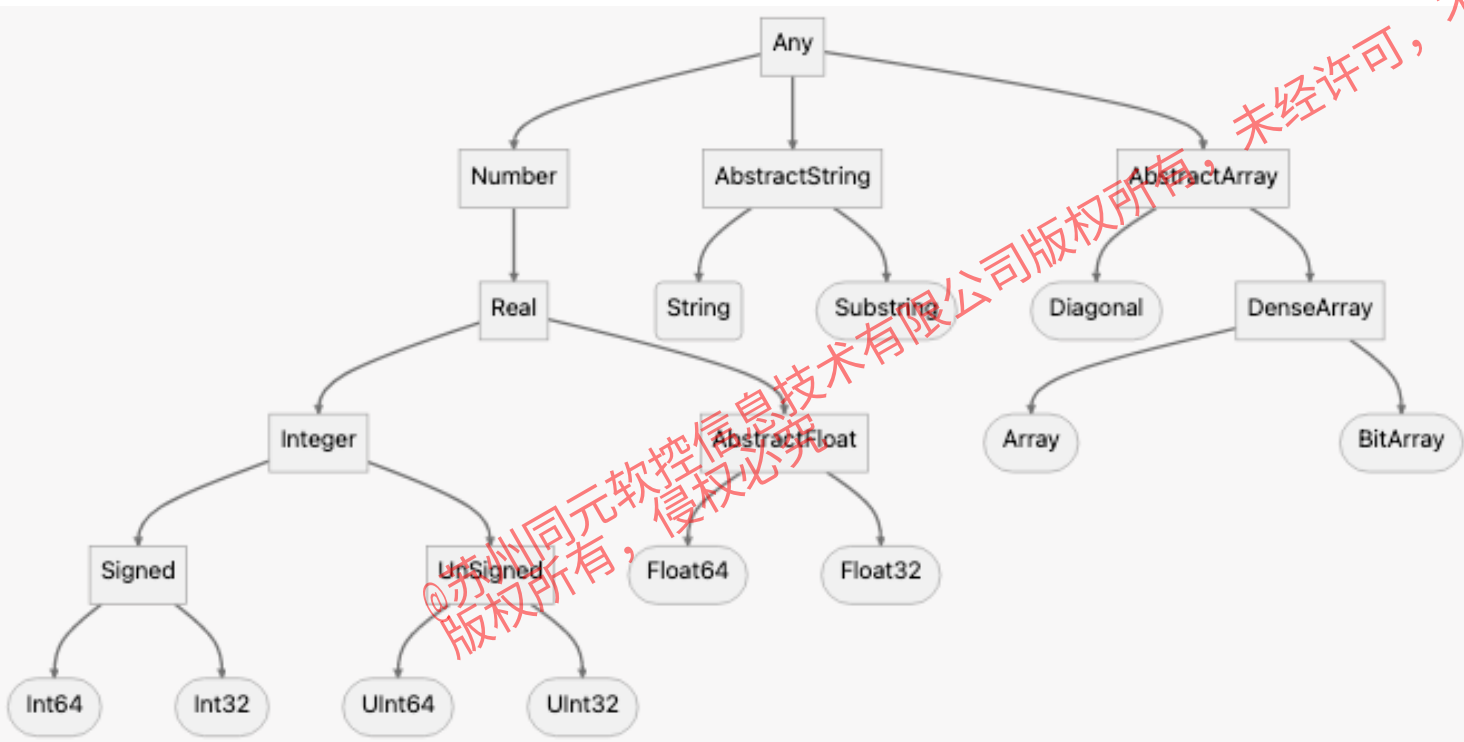
当 Julia 代码以非交互式模式执行ans时，并不会有结果，但MATALB会显示最后一个表达式的值。

```
1
2
a = 3
ans
```

```
1
2
a = 3
ans
```

2.2.1 基础知识-数据类型

整数和浮点值是算术和计算的基础。这些数值的内置表示被称作原始数值类型（numeric primitive），且整数和浮点数在代码中作为立即数时称作数值字面量（numeric literal）。例如，1 是个整型字面量，1.0 是个浮点型字面量，它们在内存中作为对象的二进制表示就是原始数值类型。



类型判断:

```
julia> eltype([1,2.0,3])  
Float64
```

```
julia> typeof([1,2.0,3])  
Vector{Float64}
```

2.2.1 基础知识-数据类型-数值类型

整数类型:

类型	是否带符号	比特数	最小值	最大值
Int8	√	8	-2^7	2^7-1
UInt8		8	0	2^8-1
Int16	√	16	-2^{15}	$2^{15}-1$
UInt16		16	0	$2^{16}-1$
Int32	√	32	-2^{31}	$2^{31}-1$
UInt32		32	0	$2^{32}-1$
Int64	√	64	-2^{63}	$2^{63}-1$
UInt64		64	0	$2^{64}-1$
Int128	√	128	-2^{127}	$2^{127}-1$
UInt128		128	0	$2^{128}-1$
Bool	N/A	8	false(0)	true(1)

```
julia> a = 1
1

julia> typeof(a) #操作系统为64位
Int64

julia> max = typemax(Int64)
9223372036854775807

julia> max + 1 #超限
-9223372036854775808

julia> min = typemin(Int64)
-9223372036854775808

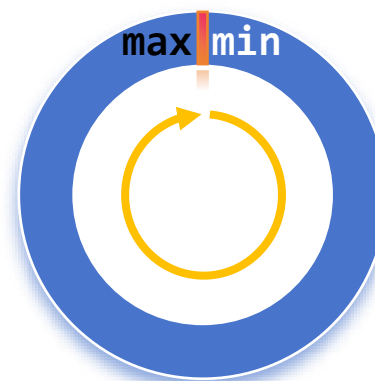
julia> min - 1 #超限
9223372036854775807

julia> x = typeof(0x123)
UInt16

julia> Int64(0x123) #转化为Int64
291
```

说明:

- 根据操作系统不同, 整数 Int 可能是 Int32 或 Int64
- 超出一个类型可表示的范围会导致环绕
- 无符号整数可以使用0x为前缀的十六进制来表示
- 可以使用T()进行数据类型转换



2.2.1 基础知识-数据类型-数值类型

浮点类型： 用于表示小数

类型	精度	比特数
Float16	半(half)	16
Float32	单(single)	32
Float64	双(double)	64

```
julia> a = 5.2
5.2

julia> typeof(a) #操作系统为64位
Float64

julia> typeof(1e5) #操作系统为64位
Float64

julia> typeof(5.2f0) #使用f则为32位
Float32

julia> Float32(a)
5.2f0
```

说明：

- 浮点数的默认类型取决于电脑系统是32位还是64位
- 浮点数可以用科学记数法表示
- 使用“f0”结尾或Float32()可以得到Float32的浮点数
- 半精度一般采用软件模拟，性能较差，不推荐使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

2.2.1 基础知识-数据类型-数值类型

复数的定义

一般形式: $a + b \cdot \text{im}$

Julia中复数的虚部用“im”表示, im为全局变量。

注意:
不能使用i和j来表示虚部

复数的构建

方法一(im直接构建): 方法二(complex函数构建):

```
Julia> 1 + 2im  
1 + 2im
```

```
julia> 1+Inf*im  
1.0 + Inf*im
```

```
julia> 1+NaN*im  
1.0 + NaN*im
```

注意:
 $a + b \cdot \text{im}$, b如果为变量名称时则错误

```
julia> a=1;b=2;complex(a,b)  
1 + 2im
```

关键字	描述
real	实部
imag	虚部
conj	复共轭
abs	绝对值
abs2	取平方后的绝对值
angle	以弧度为单位的相位角
sqrt	开根号
cos	求余弦
exp	指数运算
sinh	双曲正弦函数运算

注意:
虽然 $-1 == -1 + 0\text{im}$, 但是要对负数求平方根, 负数只能写成复数的形式。

```
julia> real(1+2im) #取实部  
1  
  
julia> imag(1+2im) #取虚部  
2  
  
julia> conj(1+2im) #求复共轭  
1 - 2im  
  
julia> abs(1+2im) #求绝对值  
2.23606797749979  
  
julia> abs2(1+2im) #取绝对值的平方  
5  
julia> sqrt(1im)  
0.7071067811865476 + 0.7071067811865475im  
  
julia> sqrt(1+2im)  
1.272019649514069 + 0.7861513777574233im  
  
julia> sqrt(-1) #负数不能直接开方, 要改为复数形式  
ERROR: DomainError with -1.0:  
sqrt will only return a complex result if  
called with a complex argument. Try  
sqrt(Complex(x))  
  
julia> sqrt(-1+0im)  
0.0 + 1.0im
```

2.2.1 基础知识-数据类型-数值类型

有理数的构建

一般形式: `a // b`, 用于表示整数精确比值的分数类型

```
julia> 2//3  
2//3
```

注意:

分子分母只能为**整型**。

有理数的标准化

- 如果一个分数的分子和分母含有公因子，它们会被约分到最简形式且分母非负
- 分数的标准化分子和分母分别可以使用 `numerator` 和 `denominator` 函数得到

```
julia> 6//9  
2//3
```

```
julia> -4//8  
-1//2
```

```
julia> numerator(4//6) #查看标准化分子  
2
```

```
julia> denominator(4//6) #查看标准化分母  
3
```

有理数的运算: `"/"` 优先级高于所有运算符(除`im`以外)

```
julia> 5//8 * 3//12  
5//32
```

```
julia> 2//7*(1+2im) #2//7优先级最高  
2//7 + 4//7*im
```

```
julia> 1//2+2im #1//2优先级最高  
1//2 + 2//1*im
```

```
julia> 1+2//3im #3im优先级最高  
1//1 - 2//3*im
```

- 使用 `float()` 将值转换为合适的浮点数类型,且完全相等
- 除`a==0`且`b==0`时, 任意整数值`a,b`从分数到浮点数转换遵循数值一致性

```
julia> float(1//2) #分数转化为小数  
0.5
```

```
julia> a=1;b=2;
```

```
julia> isequal(float(a//b),a/b)  
true
```

2.2.1 基础知识-数据类型-字符和字符串

字符与字符串声明

Julia中, Char表示单个字符, 用单引号包围
String表示字符串, 用双引号或三引号包围
Char不等于String

```
julia> 'x' #Char类型
'x': ASCII/Unicode U+0078 (category Ll: Letter, lowercase)

julia> typeof('x')
Char

julia> "中" #String类型
"中"

julia> typeof("中")
String

julia> print("""中国:"欢迎你".""")
中国:"欢迎你"。 #字符串中使用引号, 最外侧需要使用"""

julia> typeof(ans)
String

julia> 'x' == "x" #不同类型
false
```

字符与字符串运算: 基于Unicode代码进行运算

```
julia> 'A' < 'a' #Char值比较, Int64('A')=65, Int64('a')=97
true

julia> 'x' - 'a' #Char值有限运算, Int64('A')=120, Int64('a')=97
23

julia> 'A' + 1 #求解结果还是Char
'B': ASCII/Unicode U+0042 (category Lu: Letter, uppercase)

julia> s = "\u2200 x \u2203 y" #\u2200可以根据Unicode代码直接转义
"∀ x ∃ y"
```

2.2.1 基础知识-数据类型-数组

数组是存储在多维网格中对象的集合。与大多数科学计算语言一样，Julia 提供原生的数组实现。

数组可以直接用方括号来构造; 语法为 `[A, B, C, ...]` 创建一个一维数组(即一个向量)，该一维数组的元素用逗号分隔。所创建的数组中元素的类型(`eltype`) 自动由括号内参数的类型确定。如果所有参数类型都相同，则该类型称为数组的 `eltype`。如果所有元素都有相同的 promotion type，那么每个元素都由 `convert` 转换成该类型并且该类型为数组的 `eltype`。否则，生成一个可以包含任意类型的异构数组——`Vector{Any}` ;该构造方法包含字符 `[]`，此时构造过程无参数给出。

```
julia> [1,2,3] # 元素类型为 Int 的向量
3-element Vector{Int64}:
 1
 2
 3

julia> promote(1, 2.3, 4//5) # Int, Float64 以及 Rational
类型放在一起则会提升到 Float64
(1.0, 2.3, 0.8)

julia> [1, 2.3, 4//5] # 从而Float64就是这个数组的元素类型
3-element Vector{Float64}:
 1.0
 2.3
 0.8

julia> []
Any[]
```

函数	描述
<code>eltype(A)</code>	A 中元素的类型
<code>length(A)</code>	A 中元素的数量
<code>ndims(A)</code>	A 的维数
<code>size(A)</code>	一个包含 A 各个维度上元素数量的元组
<code>size(A,n)</code>	A 第 n 维中的元素数量
<code>axes(A)</code>	一个包含 A 有效索引的元组
<code>axes(A,n)</code>	第 n 维有效索引的范围
<code>eachindex(A)</code>	一个访问 A 中每一个位置的高效迭代器
<code>stride(A,k)</code>	在第 k 维上的间隔 (stride) (相邻元素间的线性索引距离)
<code>strides(A)</code>	包含每一维上的间隔 (stride) 的元组

2.2.2 基础知识-数组操作

索引读写：数组的索引读写

```
julia> A = [1 2 3; 4 5 6; 7 8 9];
julia> A[2:3] # 使用向量 j:k 对 A 进行索引
2-element Vector{Int64}:
 4
 7
julia> A[:, 2:3] # 数组的第 2 到 3 列
3×2 Matrix{Int64}:
 2  3
 5  6
 8  9
julia> A[2:3, :] # 数组的第 2 到 3 行
2×3 Matrix{Int64}:
 4  5  6
 7  8  9

# 数组的第 1 到 2 行, 第 1 到 2 列
julia> A[1:2, 1:2]
2×2 Matrix{Int64}:
 1  2
 4  5

julia> A = ones(2,2,2);
julia> A[:, :, 2] # 三维数组 A 的第 2 页
2×2 Matrix{Float64}:
 1.0  1.0
 1.0  1.0
```

1. Julia 中第一个索引为 1。
2. end 除了是终止 for、while、try、if 等语句的关键字以外，也是数组的最后一个索引。例如，X[end] 是 X 的最后一个元素，X[3:end] 选择 X 的第三个到最后一个元素。
3. A[:,n] 是矩阵 A 的第 n 列。
4. A[m,:] 是矩阵 A 的第 m 行。
5. A[:] 将 A 中的所有元素重构成一个列向量。如果 A 已经是列向量，则此表达式没有任何作用。
6. A[j:k] 使用向量 j:k 对 A 进行索引，因此相当于向量 [A[j], A[j+1], ..., A[k]]。
7. A[:,j:k] 包含第一个维度中的所有下标，但使用向量 j:k 对第二个维度进行索引。这样将返回包含列 [A[:,j], A[:,j+1], ..., A[:,k]] 的矩阵。
8. A[:,:,p] 是三维数组 A 的第 p 页。

2.2.2 基础知识-数组操作

数组的拼接

如果方括号里的参数不是由逗号分隔，而是由单个分号(;) 或者换行符分隔，那么每一个参数就不再解析为一个单独的数组元素，而是纵向拼接起来。如果这些参数是被制表符、空格符或者两个分号所分隔，那么它们的内容就横向拼接在一起。

```
julia> [1:2, 4:5] # 这里有一个逗号，因此并
不会发生矩阵的拼接。range 本身就是元素
2-element Vector{UnitRange{Int64}}:
 1:2
 4:5
```

```
julia> [1:2; 4:5]
4-element Vector{Int64}:
 1
 2
 4
 5
```

单个分号（或换行符）和空格（或制表符）可以被结合起来使用进行横向或者纵向的拼接，空格（和制表符）的优先级高于分号，首先执行纵向拼接，然后拼接结果。

```
julia> [1 2
        3 4]
2×2 Matrix{Int64}:
 1  2
 3  4
```

```
julia> [1:2 4:5 7:8]
2×3 Matrix{Int64}:
 1  4  7
 2  5  8
```

```
julia> [1 2 3] # 数字可以被横向拼接
1×3 Matrix{Int64}:
 1  2  3
```

```
julia> [1;; 2;; 3;; 4]
1×4 Matrix{Int64}:
 1  2  3  4
```

语法	函数	描述
	<code>cat(A, B, dims=k)</code>	沿着第k维拼接数组
<code>[A; B; C; ...]</code>	<code>vcat(A,B,C,...)</code>	纵向拼接
<code>[A B C ...]</code>	<code>hcat(A,B,C,...)</code>	横向拼接
<code>[A B; C D; ...]</code>	<code>hvcat(rows, A, B, C, D)</code>	同时沿垂直和水平方向拼接
<code>[A; C;; B; D;;; ...]</code>	<code>hvncat</code>	同时进行n维拼接，其中分号的数量表示拼接所在的维度

2.2.2 基础知识-数组操作

向量和矩阵的相互转化

1. 向量转矩阵

```
# 将向量转为列矩阵
v = [1, 2, 3] # Vector
column_matrix = reshape(v, :, 1) # 转为列 3x1 矩阵
# 输出: [1; 2; 3;]
```

将向量转为行矩阵

```
v = [1, 2, 3] # Vector
row_matrix = reshape(v, 1, :) # 转为 1x3 矩阵
# 输出: [1 2 3]
```

将向量转为指定形状的矩阵

```
v = [1, 2, 3, 4, 5, 6] # Vector
matrix_2x3 = reshape(v, 2, 3) # 转为 2x3 矩阵
# 输出: [1 2 3; 4 5 6]
```

2. 矩阵转向量

```
# 使用 vec 函数
A = [1 2; 3 4]
v = vec(A)
# 输出: [1, 3, 2, 4]
```

```
# 使用 reshape 函数
A = [1 2; 3 4]
v = reshape(A, :)
# 输出: [1, 3, 2, 4]
```

2.2.3 基础知识-运算符

分类	功能	符号
算数操作符	基本算数运算（包含一元操作符）	<code>+, -, √, *, /</code>
	幂运算	<code>2^3 # 8</code>
	除法	<code>3/12 # 0.25 3÷12 #0</code>
	反向除法	<code>7\3 == 3/7 # true</code>
	取余	<code>x % y 或 rem(x,y)</code>
数值比较符	取反	<code>!true # false</code>
	等于	<code>a == b</code>
	不等于	<code>a != b 或 a ≠ b</code>
	小于与大于	<code>< 与 ></code>
	小于等于	<code><= 或 ≤</code>
	大于等于	<code>>= 或 ≥</code>
向量化操作符	逐元素运算(点运算)	<code>[1, 2, 3] .+ [1, 2, 3] == [2, 4, 6] # true</code> <code>[1, 2, 3] .* [1, 2, 3] == [1, 4, 9] # true</code>
测试函数	检测非数值(NaN)	<code>isnan(NaN) # true 而不是 NaN == NaN # false</code>
三元运算符	三元运算符	<code>a == b ? "Equal" : "Not equal"</code>
位运算符	按位取反、按位与、按位或	<code>~a、a&b、a b</code>
布尔运算符	短路与 和 短路或	<code>a && b 和 a b</code>
等价符	对象等价	<code>a === b</code>

```
julia> 3÷12
#0,符号 ÷取 x / y 的整数部分, 同div(x,y)
#可以通过输入 \div<tab>打出
```

特例：直接放在标识符或括号前的数字，如 `2x` 或 `2(x+y)` 还会被视为乘法，数值字面量系数

```
julia> x=3
3
julia> 2x+3
9
```

在 Julia 中，如果 A 和 B 是数组，像 `A == B` 这样的逻辑比较运算符不会返回布尔值数组。

相反地，请使用 `A .== B`。对于其他的像 `<`、`>` 这样的比较运算符同理。

```
a = [1 2 3];
b = [4 5 6];

a == b
# false

a .== b
# 1×3 BitArray{2}:
# 0 0 0
all(a .== b)
# false
```

```
a = [1 2 3];
b = [4 5 6];

a == b
% 0 0 0

all(a == b)
% 0
```

2.2.3 基础知识-运算符

数值字面量系数

为了让常见的数值公式和表达式更清楚，Julia 允许变量直接跟在一个数值字面量后，暗指乘法。这可以让写多项式变得很清楚：

```
julia> x = 3
3

julia> 2x^2 - 3x + 1
10

julia> 1.5x^2 - .5x + 1
13.0

julia> 2^2x
64
```

数值字面量系数的优先级跟一元运算符相同，比如说取相反数。所以 2^3x 会被解析成 $2^{(3x)}$ ，而 $2x^3$ 会被解析成 $2*(x^3)$ 。

数值字面量也能作为括号表达式的系数：

```
julia> 2(x-1)^2 - 3(x-1) + 1
3
```

此外，括号表达式可以被用作变量的系数，暗指表达式与变量相乘，无论是把两个括号表达式并列，还是把变量放在括号表达式之前，都不会被用作暗指乘法

```
julia> (x-1)x
6

julia> (x-1)(x+1)
ERROR: MethodError: objects of type Int64 are not callable
julia> x(x+1)
ERROR: MethodError: objects of type Int64 are not callable
```

用于隐式乘法的数值字面量系数的优先级高于其它的二元运算符，例如乘法 ($*$) 和除法 ($/$ 、 $\backslash$ 以及 $//$)。这意味着， $1 / 2im$ 等于 $-0.5im$ 以及 $6 // 2(2+1)$ 等于 $1 // 1$ 。

2.2.3 基础知识-运算符

下表将按运算符优先级从高到低列出各个运算符。在表达式中，较高优先级的运算符会优先被计算。

分类	运算符	结合性
幂运算	$\wedge$	右结合
一元运算符	$+ - \sqrt{}$	右结合
移位运算	$<< >> >>>$	左结合
除法	$//$	左结合
乘法	$* / \% \& \backslash \div$	左结合
加法	$+ - \vee$	左结合
比较	$> < >= <= == === != !== <$	无结合性
流程控制	$\&\&$ followed by $ $ followed by $?$	右结合
Pair 操作	$=>$	右结合
赋值	$= += -= *= /= //= \wedge= \div= \% = = \&= \vee= <=> >=> >>=>$	右结合

```
a=2;b=1;c=3
d=a^b^c
#2
```

```
>> a=2;b=1;c=3;
d=a^b^c

d =

8
```

可以通过内置函数 `Base.operator_precedence` 查看任何给定运算符的优先级数值，数值越大优先级越高：

```
Base.operator_precedence(:+), Base.operator_precedence(:*),
Base.operator_precedence(:.)
#(11, 12, 17)
```

2.2.3 基础知识-运算符-广播

Julia中标量类型的函数

函数	描述
sin	正弦函数
cos	余弦函数
tan	正切函数
sqrt	平方根函数
abs	绝对值/模函数
Float64	双精度类型
Real	实数类型

标量类型的函数，对数组使用
需要加广播 (.)。

Julia中常见支持数组输入的函数

函数	描述
maximum	最大值函数
sum	求和函数
real	求实部函数

Matlab常用的向量化函数

函数	描述
all	确定所有数组元素均为非零元素还是true
any	确定是否有任何数组元素非零
cumsum	累积和
max	最大值 (注, Julia中为maximum)
find	查找非零元素的索引和值
sort	对数组元素排序 (注, Julia中获取排序后的索引使用sortperm)
sum	数组元素求和

需要注意的是matlab中的这些函数例如sum、max等，如果对一个矩阵进行运算，它返回的默认是第一个维度上计算的结果，等价于julia的sum(x;dims=1)，而Julia中默认的是返回整个矩阵进行函数运算的结果，而matlab要在数据后加上"all"标记。这一点，大家上手用的时候可能会有些不习惯

2.2.3 基础知识-运算符-广播

向量化的“点”运算符

用途：计算元素或向量的结果；操作数为数组或标量

说明：算数运算符和复合运算符都有相应的点操作

! $\sqrt{\quad}$ += -= *= /= \= %= ^= &= |= $\underline{\quad}$ = >>>= >>= <<=

```
A,B,B1=[1,2,3],[4,5,6],2
A ^ 3      #错误：未定义
A .^ 3     #定义为计算元素（或“向量化”）结果[1^3, 2^3, 3^3]
```

```
# 3-element Array{Int64,1}:
# 1
# 8
# 27
```

```
A .^ B     #A .^ B==(^).(A,B)
# 3-element Array{Int64,1}:
# 1
# 32
# 729
```

A .^ B1#执行广播操作

```
# 3-element Array{Int64,1}:
# 1
# 4
# 9
```

```
2 .* A.^2 .+ sin.(A)#等效:@. 2A^2 + sin(A) 用公式2a^2 + sin(A)对A中的每个元素计算
```

```
# 3-element Array{Float64,1}:
# 2.8414709848078967
# 8.909297426825681
# 18.14112000805987
```

```
# 点运算符与数字文字的组合可能是不明确的(如1.+A为1. + A)
# 在这种情况下必须在运算符周围使用空格（在1后面加空格）
```

```
1 .+A
# 3-element Array{Int64,1}:
# 2
# 3
# 4
```

=====

```
# 操作符 .< 是特别针对数组
```

```
A,B=[1,4,7],[2,3,8]
```

```
0 .< A .< B      #只有当 A 和 B 有着相同的大小时，A .< B 才是合法
```

```
# 3-element BitArray{1}:结果对应布尔数组，满足条件元素返回 true
# 1
# 0
# 1
```

2.2.3 基础知识-运算符-复合赋值运算符

Julia中的每个二元运算符和位运算符都可以与赋值符号“=”联用，可以称之为复合赋值运算符，联用的含义指把运算的结果再赋予参与运算的变量。例如：

```
julia> x = 5; x %= 3
```

```
2
```

```
julia> x = 5; x += 3
```

```
8
```

因为这种更新运算相当于把新的值与原有的变量进行绑定，所以原有变量的类型可能会因此发生改变。

复合赋值运算符有下面几种：

`+=` `-=` `*=` `/=` `\=` `÷=` `%=` `^=` `&=` `|=` `⊔=` `>>>=` `>>=` `<<=`

2.3.1 常用函数-舍入函数

函数	描述	返回类型
round(x)	把 x 舍入到最近的小数或整数	Float64/Int64
floor(x)	把 x 向 -Inf 取整	Float64/Int64
ceil(x)	把 x 向 +Inf 取整	Float64/Int64
trunc(x)	把 x 向 0 取整	Float64/Int64

```
round(0.5) # Julia返回0, Matlab返回1  
round(1.5) # Julia返回2, Matlab返回2
```

Julia中，round(X)按照默认的四舍五入模式将X舍入为整数值，并返回与X相同类型的值。默认情况下这将四舍五入到最近的整数，如果出现小数值为0.5，则会被舍入为最近的**偶数整数**。

Matlab中，round(X) 将 X 的每个元素四舍五入为最近的整数。在对等情况下，即有元素的小数部分恰为 0.5 时，round 函数会偏离零四舍五入到具有更大幅值的整数。

2.3.2 常用函数-除法函数、符号函数和绝对值函数

函数	描述
div(x,y)	截断取整除法; 商向 0 舍入
fld(x,y)	向下取整除法; 商向 -Inf 舍入, 等同于 div(x,y, RoundDown)
cld(x,y)	向上取整除法; 商向 +Inf 舍入, 等同于 div(x,y, RoundUp)
rem(x,y)	除法余数;
divrem(x,y)	返回 (div(x,y),rem(x,y))
mod(x,y)	取模余数;
mod2pi(x)	对 2pi 取模余数; $0 \leq \text{mod2pi}(x) < 2\pi$
gcd(x,y,...)	x, y, ... 的最大公约数, 非负数
lcm(x,y,...)	x, y, ... 的最小公倍数, 非负数

```
rem(21,4)
#1
rem(-21,4)
#-1
```

@苏州同元软控信息技术有限公司版权所有，侵权必究

函数	描述
abs(x)	x 的绝对值
abs2(x)	x 的绝对值的平方
sign(x)	x 的正负号, 返回值为 -1, 0, 或 +1
signbit(x)	是否有符号位, 有 (true) 或者 无 (false)
copysign(x,y)	返回一个数, 它具有 x 的绝对值, y 的符号位
flipsign(x,y)	返回一个数, 它具有 x 的绝对值, x*y 的符号位

```
V = [-11, 0, 1.5, Inf, NaN]
W = [Inf, NaN, 0, -1, -1]
copysign(V, W)
#5-element Vector{Float64}:
# 11.0
# 0.0
# 1.5
# -Inf
# NaN
# 0、Inf 和 NaN 都不取出负号; Inf 在复制负号后变为 -Inf, 而 NaN 在复制负号后不变
```

2.3.3 常用函数-乘方，对数和开方函数

函数	描述
<code>sqrt(x)</code>	x 的平方根
<code>cbrt(x)</code>	x 的立方根
<code>hypot(x,y)</code>	误差较小的 $\sqrt{x^2 + y^2}$
<code>exp(x)</code>	自然指数 e 的 x 次幂
<code>expm1(x)</code>	当 x 接近 0 时, 精确计算 $\exp(x)-1$
<code>ldexp(x,n)</code>	当 n 为整数时, 高效计算 $x*2^n$
<code>log(x)</code>	x 的自然对数
<code>log(b,x)</code>	以 b 为底 x 的对数
<code>log2(x)</code>	以 2 为底 x 的对数
<code>log10(x)</code>	以 10 为底 x 的对数
<code>log1p(x)</code>	当 x 接近 0 时, 精确计算 $\log(1+x)$
<code>exponent(x)</code>	<code>trunc(log2(x))</code>
<code>significand(x)</code>	returns the binary significand (a.k.a. mantissa) of a floating-point number x

$\log M(N) = \log(N) / \log(M)$

为什么要有 `hypot`, `expm1`, `log1p` 等函数, 参见 John D. Cook 的博客: [What's so hard about finding a hypotenuse? \(johndcook.com\)](http://johndcook.com/What's%20so%20hard%20about%20finding%20a%20hypotenuse/)

2.3.4 常用函数-三角函数和双曲函数

Julia 内置了所有的标准三角函数和双曲函数

sin	cos	tan	cot	sec	csc
sinh	cosh	tanh	coth	sech	csch
asin	acos	atan	acot	asec	acsc
asinh	acosh	atanh	acoth	asech	acsch
sinc	cosc				

```
julia> atan(4,3)
0.9272952180016122
```

```
julia> atan(-4,-3)
-2.214297435588181
```

```
julia> atan((-4)/(-3))
0.9272952180016122
```

```
julia> atan(4/3)
0.9272952180016122
```

需要注意的是：julia中的四象限反正切用的函数也是 `atan(x,y)`

另外，`sinpi(x)`和 `cospi(x)`各自被提供给更准确的 `sin(pi*x)`和 `cos(pi*x)`的计算

如果想要以度，而非弧度，为单位计算三角函数，应使用带 `d` 后缀的函数。例如，`sind(x)` 计算 `x` 的正弦值，这里 `x` 的单位是度。以下的列表是全部的以度为单位的三角函数：

sind	cosd	tand	cotd	secd	cscd
asind	acosd	atand	acotd	asecd	acscd

2.3.5 常用函数-数据分析函数

名称	含义	名称	含义	名称	含义
minimum	最小值	prod	数组元素的乘积	diff	差分、近似导数
mean	平均值（迭代器内）	cumsum	累计和	length	个数
std	标准差	cross, x	叉积	sum	数组元素总和
sort	排序	maximum	最大值	dot, ·	点积
norm	欧氏长度、模	median	中位数	cumprod	累计乘积
middle	找到数组中最大值和最小值，计算他们的平均值				

```
maximum([1,2,3])#3
max(1,2,3)#3
```

```
middle([1,2,4,5,9])
#5.0
```

maximum返回集合的最大值

max返回 timeseries 对象中数据样本的最大值

2.3.5 常用函数-数据分析函数

在 Julia 中，像 sum、prod 和 max 等函数，以单参数调用时，即使数组 A 是多维数组，规约操作也会逐元素进行。

```
a = [1 2 3; 4 5 6]
# 2×3 Array{Int64,2}:
#  1  2  3
#  4  5  6
```

```
sum(a)
# 21
sum(a; dims=1)
# 1×3 Array{Int64,2}:
#  5  7  9
```

```
prod(a)
# 720
prod(a; dims=1)
# 1×3 Array{Int64,2}:
#  4 10 18
```

```
max(a...)
# 6
maximum(a; dims=1)
# 1×3 Array{Int64,2}:
#  4  5  6
```

M软件

```
a = [1 2 3; 4 5 6]
% 1 2 3
% 4 5 6
```

```
sum(a)
% 5 7 9
```

```
prod(a)
% 4 10 18
```

```
max(a)
% 4 5 6
```

在 Julia 中，集合的元素可以使用 splat 运算符 ... 来作为参数传递给函数，如max (a...)

03

PART 03 ➔

脚本构建与代码调试

@苏州同元软件技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

3.1.1 脚本文件与函数文件

Syslab中有一种文件叫jl文件，他就是一系列代码组成的一个扩展名为.jl的文件，和其语法和高级语言一样，是一种程序化的编程语言。

- jl文件可分为脚本文件（Syslab scripts）和函数文件（Syslab functions）
- 脚本文件是包含多条Syslab命令的文件；函数文件可以包含输入变量，并把结果传送给输出变量

```
FunTest.jl x vanderpoldemo.jl
1 # 初始值问题
2 using TyMath
3 using TyPlot
4 #直接写函数
5 # function vanderpoldemo(t,y,Mu)
6 #     return [y[2]; Mu*(1-y[1]^2)*y[2]-y[1]]
7 # end
8
9 tspan = [0 20];
10 y0 = [2; 0];
11 Mu = 1;
12 #调用函数
13 include("vanderpoldemo.jl")
14 ode = (t,y)->vanderpoldemo(t,y,Mu);
15 t,y = ode45(ode, tspan, y0);
16
17 plot(t,y[:,1])
18 xlabel("t")
19 ylabel("solution y")
20 title("van der Pol Equation,  $\mu = 1$ ")
```

```
FunTest.jl vanderpoldemo.jl x
1 function vanderpoldemo(t,y,Mu)
2     return [y[2]; Mu*(1-y[1]^2)*y[2]-y[1]]
3 end
```

```
# 初始值问题
using TyMath
using TyPlot
#直接构造函数
# function vanderpoldemo(t,y,Mu)
#     return [y[2]; Mu*(1-y[1]^2)*y[2]-y[1]]
# end
tspan = [0 20];
y0 = [2; 0];
Mu = 1;
#调用函数
include("vanderpoldemo.jl")
ode = (t,y)->vanderpoldemo(t,y,Mu);
t,y = ode45(ode, tspan, y0);

plot(t,y[:,1])
xlabel("t")
ylabel("solution y")
title("van der Pol Equation,  $\mu = 1$ ")
```

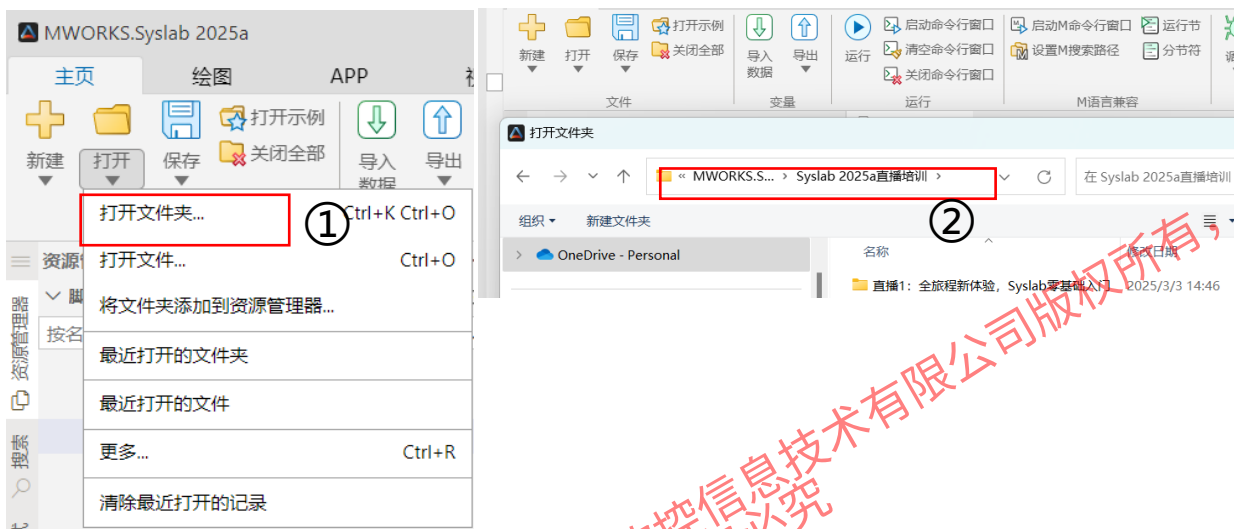
```
function vanderpoldemo(t,y,Mu)
    return [y[2]; Mu*(1-y[1]^2)*y[2]-y[1]]
end
```

3.1.2 文件管理

打开相关文件夹作为工作目录，并在资源管理器中管理文件。

操作步骤：

- 选择工作目录：主页-打开-“打开文件夹”
- 选择已有文件夹



Tips: 新建Python或M脚本，Syslab可以使用Python或M环境，Syslab内置Python环境版本为3.7.5



➢ 新建脚本文件-sun

- 工作目录下新建脚本：M或者julia

- 工作目录下新建文件夹

➢ 点击文件右键对文件进行重命名

➢ 不需要的脚本可以delete或右击选择删除

➢ 返回上一级：可以返回上级文件夹

3.1.2 文件管理

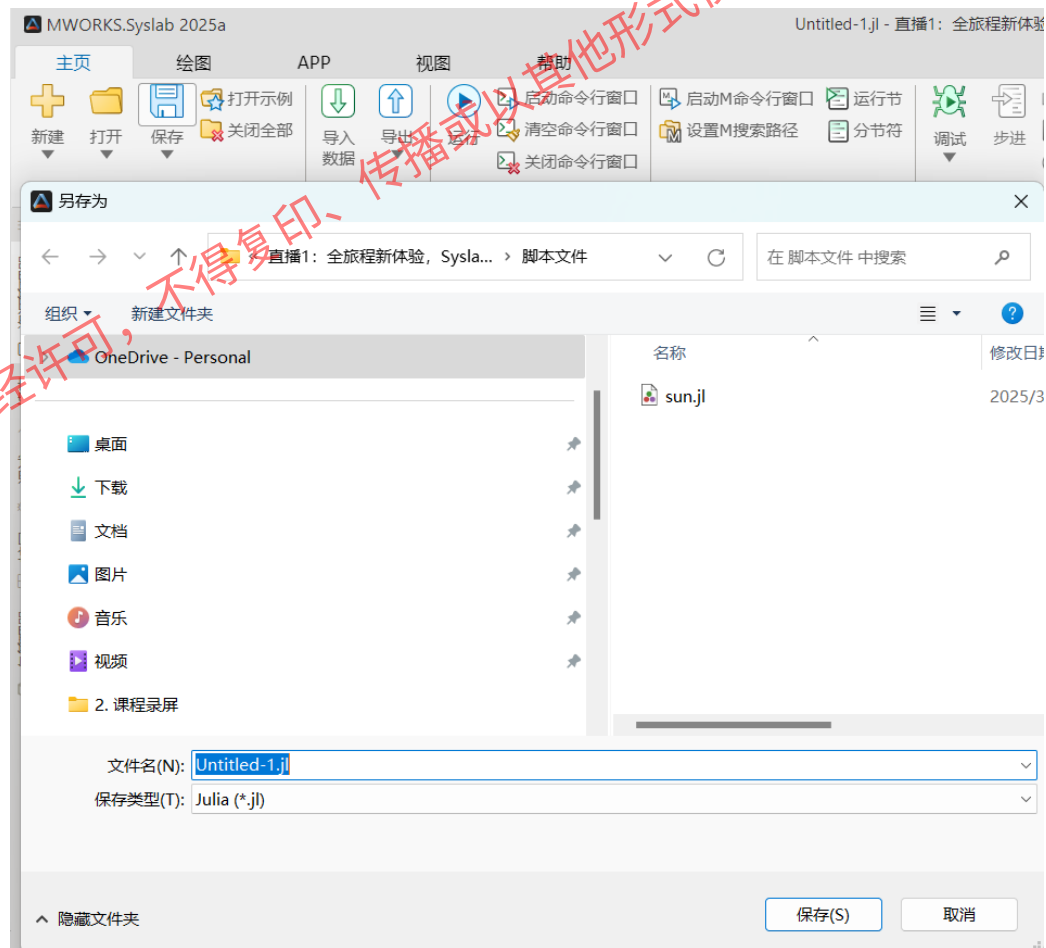
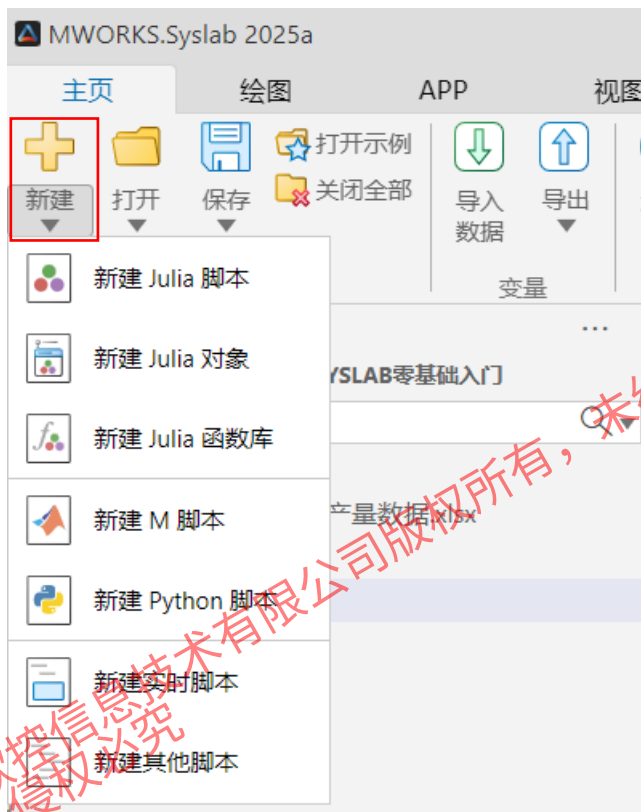
打开相关文件夹作为工作目录，并在资源管理器中管理文件。

操作步骤：

- “新建” 选择文件格式

Tips: 新建Python或M脚本, Syslab可以使用Python或M环境, Syslab内置Python环境版本为3.7.5

- “保存” 选择文件夹进行重命名保存



3.1.3 代码编辑

#加载工具箱

```
using TyPlot
```

```
using TyMath
```

#定义变量

```
dec = 23.4 # 太阳偏角: 23° 24'
```

```
lat = 31 + 30 / 60 # 纬度: 31° 30'
```

#使用函数转换弧度

```
dec = deg2rad(dec) # 将角度转换为弧度
```

```
lat = deg2rad(lat)
```

#定义时间变变量

```
# 默认时间从早上5点开始, 到19点结束
```

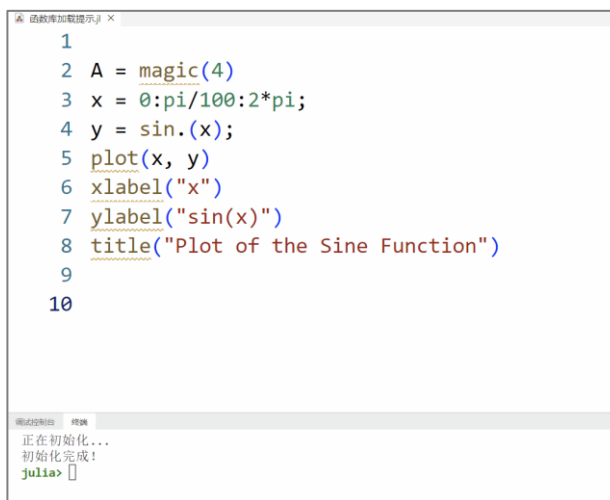
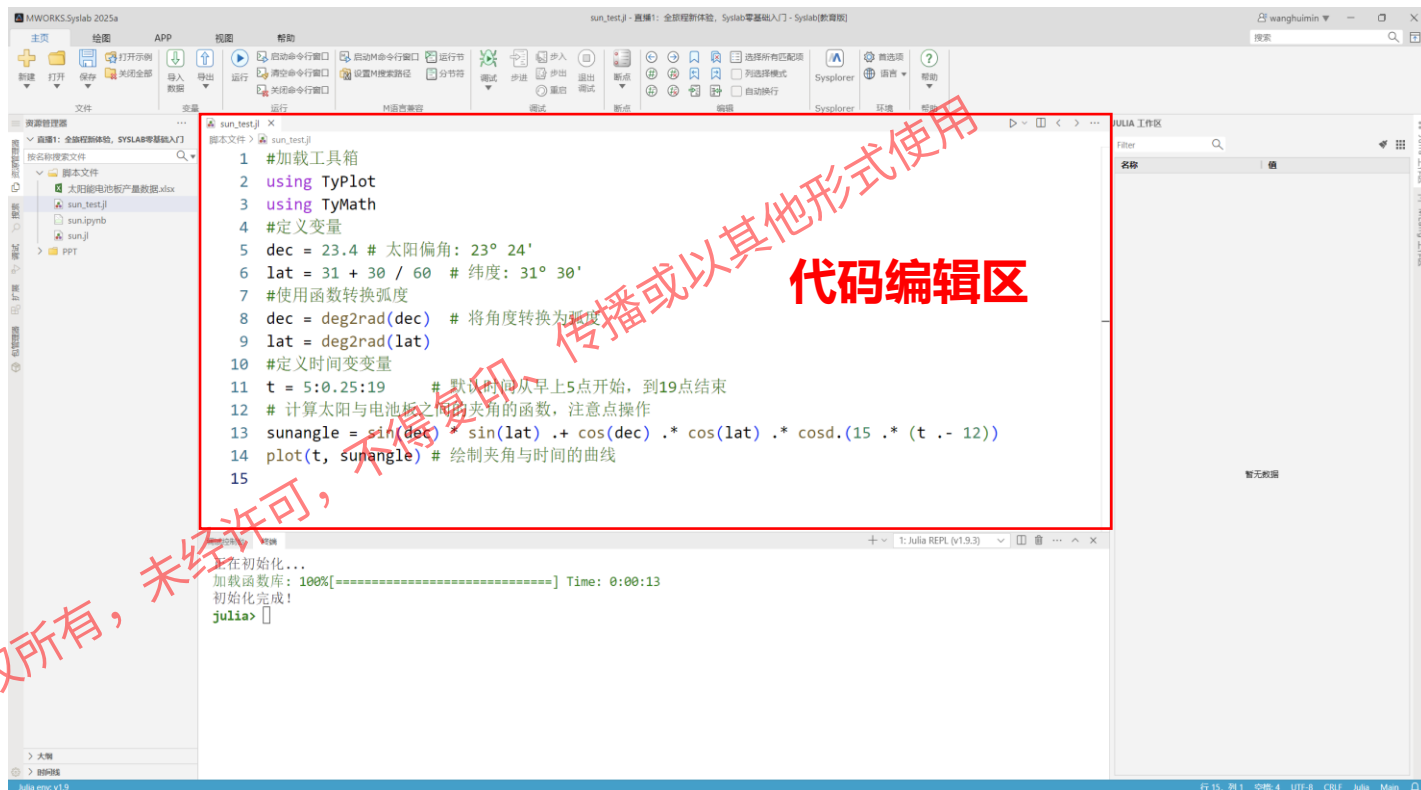
```
t = 5:0.25:19
```

计算太阳与电池板之间的夹角的函数, 注意点操作

```
sunangle = sin(dec) * sin(lat) .+ cos(dec) .*  
cos(lat) .* cosd.(15 .* (t .- 12))
```

#调用绘图函数进行图形绘制

```
plot(t, sunangle) # 绘制夹角与时间的曲线
```



函数库加载提示:

- 对于从资源管理器中打开的Julia脚本, 会检查该文件中未定义的库函数, 以波浪线提示, 并提供快速修复功能

3.1.4 代码运行

运行当前文件

- 主页工具栏运行按钮
- 快捷键：Ctrl+F5

运行部分代码

方法一

①：左键选中需要运行的代码段落，例如阴影部分

②：使用：Ctrl+enter

方法二

①：代码前用##标识/分节符

②：运行当前节

MWORKS.Syslab 2025a sun.jl - 直播1: 全流程新体验, Syslab零基础入门 - Syslab[教育版] wanghuimin

主页 绘图 APP 视图 帮助

新建 打开 保存 打开示例 关闭全部 导入数据 导出 运行 启动命令行窗口 清空命令行窗口 关闭命令行窗口 启动M命令行窗口 设置M搜索路径 运行节 分节符 调试 步进 步出 退出 断点 选择所有四项 列选择模式 自动换行

资源管理器 脚本文件

- 直播1: 全流程新体验, SYSLAB零基础入门
- 按名称搜索文件
- 脚本文件
 - 太阳能电池板产量数据.xlsx
 - sun.ipynb
 - sun.jl
 - PPT

脚本文件 > sun.jl

```
1 ##
2 dec = 23.4 # 太阳偏角: 23° 24'
3 lat = 31 + 30 / 60 # 纬度: 31° 30'
4 dec = deg2rad(dec) # 将角度转换为弧度
5 lat = deg2rad(lat)
6 t = 5:0.25:19 # 默认时间从早上5点开始, 到19点结束
7 # 计算太阳与电池板之间的夹角的函数
8 sunangle = sin(dec) * sin(lat) .+ cos(dec) .* cos(lat) .* cosd.
9 plot(t, sunangle) # 绘制夹角与时间的曲线
10
11 T_inc = 1.4883 * 0.7 .^ (sunangle .^ -0.678) # 考虑大气层影响
12 production_theory = 270 * T_inc .* sunangle # 计算电池板理论产量
13 k = find(production_theory .> 207) # 考虑电池板最大输出为207kw
14 production_theory[k] .= 207
15 plot(t, production_theory) # 绘制理论产量与时间的曲线
16 xlabel("时间") # 添加X轴标签
```

调试控制台 终端

```
[0x7FFC1A4EAF90] ANOMALY: meaningless REX prefix used
[0x7FFBF105E180] ANOMALY: meaningless REX prefix used
[0x7FFC19BBC0B0] ANOMALY: meaningless REX prefix used
[0x7FFC1A18FD80] ANOMALY: use of REX.w is meaningless (default operand size is 64)
[0x7FFC1A188D80] ANOMALY: use of REX.w is meaningless (default operand
```

Julia env: v1.9

Julia 工作区

Filter

名称	值
dec	0.40840704496667307
err	1-element Base.ExceptionStack
k	23-element Vector{Int64}
lat	0.5497787143782138
production_theory	57-element Vector{Float64}
sunangle	57-element Vector{Float64}
t	5.0:0.25:19.0
T_inc	57-element Vector{Float64}

Figure 1

理论数据

产量(kW)

时间

示例见：本课件第6章节示例

3.2.1 控制流

if –else-end结构

if 语句可以根据布尔表达式的值，让部分代码被执行或者不被执行。

test.jl

```
function test(x, y)
    if x < y
        println("x is less than y")
    elseif x > y
        println("x is greater than y")
    else
        println("x is equal to y")
    end
end
```

Julia REPL

```
julia> include("test.jl")
julia> test(1, 2)
x is less than y
julia> test(2, 1)
x is greater than y
julia> test(1, 1)
x is equal to y
```

1. 如果表达式 $x < y$ 是 true，那么对应的代码块会被执行；否则判断条件表达式 $x > y$ ，如果它是 true，则执行对应的代码块；如果没有表达式是 true，则执行 else 代码块。
2. elseif 和 else 代码块是可选的，并且可以使用任意多个 elseif 代码块。

3.2.1 控制流

if-else-end结构

```
function test(x,y)
    if x < y
        relation = "less than"
    elseif x == y
        relation = "equal to"
    else
        relation = "greater than"
    end
    println("x is ", relation, " y.")
end
```

变量 relation 是在 if 代码块内部声明的，但可以在外部使用。然而，在利用这种行为的时候，要保证变量在所有的分支下都进行了定义。否则会导致运行时错误。

```
julia> include("test.jl")

julia> test(1, 2)

x is less than y

julia> test(2, 1)

x is greater than y

julia> test(1, 1)

x is equal to y
```

1. 在 Julia 中的 if 语句中，一个条件表达式的值如果不是 true 或者 false 的话，会返回错误。
2. 在 Julia 中，**并没有直接的 switch case 语句**，但你可以使用多个 if-elseif-else 语句

```
julia> if 1
    println("true")
end

ERROR: TypeError: non-boolean (Int64) used
in boolean context
```

3.2.1 控制流

三元运算符

所谓的 "三元运算符", `?:`, 很类似 if-else 语法, 它用于选择性获取单个表达式的值, 而不是选择性执行大段的代码块。

```
a ? b : c
```

在 `?` 之前的表达式 `a`, 是一个条件表达式, 如果条件 `a` 是 `true`, 三元运算符计算在 `:` 之前的表达式 `b`; 如果条件 `a` 是 `false`, 则执行 `:` 后面的表达式 `c`。注意, `?` 和 `:` 旁边的空格是强制的。重要地是, 与 if-else 类似, `:` 之前和之后的表达式只有在条件表达式为 `true` 或者 `false` 时才会被相应地执行。

```
julia> x = 1; y = 2;  
julia> println(x < y ? "less than" : "not  
less than")  
less than  
  
julia> x = 1; y = 0;  
julia> println(x < y ? "less than" : "not  
less than")  
not less than
```

如果表达式 `x < y` 为真, 整个三元运算符会执行字符串 "less than", 否则执行字符串 "not less than"。

3.2.1 控制流

While、for结构

while 循环会执行条件表达式，只要它为 true，就一直执行while 循环的主体部分。当 while 循环第一次执行时，如果条件表达式为 false，那么主体代码就一次也不会被执行。for 循环使得常见的重复执行代码写起来更容易。 while 循环中用到的向上和向下计数是可以用 for 循环更简明地表达。

```
julia> i = 1;
julia> while i <= 5
    println(i)
    global i += 1
end
```

```
1
2
3
4
5
```

```
julia> for i = 1:5
    println(i)
end
```

```
1
2
3
4
5
```

这里的 1:5 是一个范围对象，代表数字 1, 2, 3, 4, 5 的序列。for 循环在这些值之中迭代，对每一个变量 i 进行赋值。for 循环与之前 while 循环的一个重要区别是作用域，即变量的可见性。如果变量 i 没有在另一个作用域里引入，在 for 循环内，它就只在 for 循环内部可见，在外部和后面均不可见。

3.2.1 控制流

While、for结构

一般来说，for 循环组件可以用于迭代任一个数据容器。在这种情况下，相比 `=`，关键字 **in** 或者 `∈` 更常用，因为它使得代码更清晰：

```
julia> for i in [1,4,0]
           println(i)
       end

1
4
0
```

```
julia> for s ∈ ["foo","bar","baz"]
           println(s)
       end

foo
bar
baz
```

©苏州同元软控信息技术有限公司版权所有，侵权必究

3.2.1 控制流

While、for结构

为了方便，我们可能会在条件不成立之前终止一个 while 循环，或者在访问到迭代对象的结尾之前停止一个 for 循环，这可以用关键字 **break** 来完成：

```
julia> while true
    println(i)
    if i >= 5
        break
    end
    global i += 1
end
```

```
1
2
3
4
5
```

```
julia> for j = 1:1000
    println(j)
    if j >= 5
        break
    end
end
```

```
1
2
3
4
5
```

对于多个嵌套的 for 循环合并成的外部循环，循环里面使用 break 语句则会跳出整个嵌套循环，不仅仅是内层循环。

3.2.1 控制流

While、for结构

在某些场景下，需要直接结束此次迭代，并立刻进入下次迭代，**continue** 关键字可以用来完成此功能：

```
julia> i = 1;
julia> while i <= 10
    global i += 1
    if i % 3 != 0
        continue
    end
    println(i)
end

3
6
9
```

```
julia> for i = 1:10
    if i % 3 != 0
        continue
    end
    println(i)
end

3
6
9
```

@苏州同元软控信息技术有限公司版权所有，侵权必究

3.3.1 函数文件定义

- 脚本文件可以理解为简单的JL文件，脚本文件中的变量都是全局变量。函数文件是在脚本文件的基础上多添加了一行函数定义行，其代码组织结构和调用方式与对应的脚本文件截然不同。
- 函数文件是以函数声明行 **"function..."** 作为开始的，其实质就是用户往Syslab函数库里边添加了子函数，函数文件中的变量都是局部变量，除非使用了特别声明。函数运行完毕之后，其定义的变量将从工作区间中清除。

关于函数文件名：函数文件名与函数名也可以不相同（一般设为相同！）。

```
""""  
函数注释
```

```
""""  
function functionname(args)  
    expression  
    ...  
    expression  
end
```

```
function 输出形参表=函数名(输入形参表)  
    注释说明部分  
    函数体语句  
end
```

```
julia> function f(x, y)  
            return x + y  
        end
```

```
julia> f(2, 3)  
5
```

简化写法：

```
julia> f(x, y) = x + y
```

```
julia> f(2, 3)  
5
```

默认情况下，函数返回的值是最后计算的表达式的值，所以上面是没有 return 语句的，当然，如果使用 return 关键字，函数就会立即返回

3.3.2 函数的输入参数

参数类型声明

可以通过将 `::TypeName` 附加到参数名称来声明函数参数的类型。

例如，下面的函数递归地计算斐波那契数：

```
fib(n::Integer) = n ≤ 2 ? one(n) : fib(n-1) + fib(n-2)
```

参数类型声明通常对性能没有影响，在 Julia 中声明参数类型最常见的原因是：

- **正确性：** 如果函数只为某些参数类型返回正确的结果，则类型声明会很有用。例如，如果我们省略参数类型并写成 `fib(n) = n ≤ 2 ? one(n) : fib(n-1) + fib(n-2)`，然后 `fib(1.5)` 会悄悄地给我们无意义的答案 1.0。
- **派发：** 对于不同的参数类型，你可以有不同版本（“方法”）的函数，在这种情况下，参数类型用于确定调用哪个版本的函数。例如，你可以实现一个完全不同的算法 `fib(x::Number) = ...`，该算法扩展为了非整数值，适用于任何 `Number` 类型。
- **清晰性：** 类型声明可以作为一种关于预期参数的文档形式。

3.3.2 函数的输入参数

变参函数

你可以通过在最后一个参数后增加省略号来定义一个变参函数:

```
# 变量 a 和 b 和以前一样被绑定给前两个参数, 后面的参数整个做为迭代集合被绑定到变量 x 上, x 为一个元组。
```

```
bar(a, b, x...) = (a, b, x)
```

```
# bar (generic function with 1 method)
```

```
bar(1, 2)
```

```
# (1, 2, ())
```

```
bar(1, 2, 3)
```

```
# (1, 2, (3,))
```

```
bar(1, 2, 3, 4)
```

```
# (1, 2, (3, 4))
```

```
bar(1, 2, 3, 4, 5, 6)
```

```
# (1, 2, (3, 4, 5, 6))
```



```
julia> bar(1, 2)  
(1, 2, ())
```

```
julia> bar(1, 2, 3)  
(1, 2, (3,))
```

```
julia> bar(1, 2, 3, 4)  
(1, 2, (3, 4))
```

```
julia> bar(1, 2, 3, 4, 5, 6)  
(1, 2, (3, 4, 5, 6))
```

3.3.2 函数的输入参数

可选参数

可以为函数参数提供合理的默认值。这可以避免用户在每次调用时必须传递每个参数:

```
# 分别给变量 b, c 默认值为 2, 3, 当没有显式的传入参数 b  
或 c 时, b 的值为 2, c 的值为 3。
```

```
f(a, b=2, c=3) = a + b + c
```

```
# f (generic function with 1 method)
```

```
f(1) # 等价于 f(1, 2, 3)
```

```
# 6
```

```
f(1, 10) # 等价于 f(1, 10, 3)
```

```
# 14
```

```
f(1, 10, 10)
```

```
# 21
```

```
julia> f(a, b=2, c=3) = a + b + c  
f (generic function with 3 methods)
```

```
julia> f(1)  
6
```

```
julia> f(1, 10)  
14
```

```
julia> f(1, 10, 10)  
21
```

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

3.3.2 函数的输入参数

关键字参数

在 Julia 中，关键字参数 (keyword arguments) 允许你在调用函数时通过参数名指定参数值，使得代码更具可读性和灵活性。关键字参数可以使这些复杂的接口更容易使用和扩展，因为它们允许通过名称而不是通过位置来标识参数，具有关键字参数的函数在签名中**使用分号定义**：

```
function myplot(x, y; style="solid", width=1, color="black")  
    ###  
end
```

例如上述定义的一个绘制直线的函数图。这个函数有很多选项，用于控制线条样式、宽度、颜色。如果我们希望只修改宽度和颜色，可以这样调用： `myplot(x, y, color = "red", width = 2)`。这样做有两个目的：

1. 这个调用更容易阅读，因为我们可以用它的含义来标记一个参数；
2. 可以以任意顺序传递大量参数的任意子集。

```
function myprint(a::String, b::String; Delimiter::String  
= " , ", EndOfLine::String = "\n")  
    println(a * Delimiter * b * EndOfLine)  
end  
  
myprint("Hello", "Julia", Delimiter = " ")
```

```
julia> function myprint(a::String, b::String; Delimiter::String=" , ", EndOfLine::String="\n")  
    println(a * Delimiter * b * EndOfLine)  
end  
myprint (generic function with 1 method)  
  
julia> myprint("Hello", "Julia", Delimiter=" ")  
Hello Julia
```

3.3.3 函数的返回值

return 关键字

默认情况下，返回值函数定义体中的最后一个表达式。

在函数 `f` 中，返回表达式 `x + y` 的值：

```
function f(x, y)
    x + y
end
```

```
f(1, 2)
```



```
julia> function f(x, y)
           x + y
       end
f (generic function with 1 method)

julia> f(1, 2)
3
```

与许多其他语言中一样，`return` 关键字会导致函数立即返回，提供一个返回值的表达式：

```
function g(x, y)
    return x + y
    x * y
end
```

```
g(1, 2)
```



```
julia> function g(x, y)
           return x + y
           x * y
       end
g (generic function with 1 method)

julia> g(1, 2)
3
```

3.3.3 函数的返回值

返回值类型声明

与输入参数的类型声明类似，可以使用 `::TypeName` 在函数声明中指定返回类型。这会将返回值转换为指定的类型。

```
function g(x, y)::Int8
    return x * y
end

typeof(g(1, 2))
```



```
julia> function g(x, y)::Int8
           return x * y
       end
g (generic function with 1 method)

julia> typeof(g(1, 2))
Int8
```

返回 nothing

对于不需要任何返回值的函数，Julia 中的写法为返回值 `nothing`：

```
function printx(x)
    println("x = $x")
    return nothing
end

r = printx("hello!")
println(r === nothing)
```



```
julia> function printx(x)
           println("x = $x")
           return nothing
       end
printx (generic function with 1 method)

julia> r = printx("hello!")
x = hello!

julia> println(r === nothing)
true
```

3.3.3 函数的返回值

多返回值

可以通过返回元组或其他可迭代值来从函数返回多个值。例如，下面的函数返回两个值:

```
function foo(a, b)
    a + b, a * b
end
```

如果在交互式会话中调用它，而没有任何地方分配返回值，您将看到元组返回

```
foo(2, 3)
```

将每个值提取到一个变量中:

```
x, y = foo(2, 3)
```

```
julia> function foo(a, b)
           a + b, a * b
       end
foo (generic function with 1 method)
julia> foo(2, 3)
(5, 6)

julia> x, y = foo(2, 3)
(5, 6)

julia> x
5

julia> y
6
```

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他形式使用
版权所有，侵权必究

3.3.4 操作符函数

操作符函数

在 Julia 中，大多数操作符只不过是支持特殊语法的函数（&& 和 || 等具有特殊评估语义的操作符除外），因此，你也可以使用带括号的参数列表来使用它们，就和任何其他函数一样：

```
julia> 1 + 2 + 3
6

julia> +(1,2,3)
6
```

你可以对操作符，例如 + 和 *，进行赋值和传参，就像其它函数传参一样：

```
#接上例
julia> f = +;

julia> f(1,2,3)
6
```

3.3.5 匿名函数

匿名函数

函数可以用作参数，也可以当作返回值。函数也可以不带函数名称地匿名创建，使用语法如下：

```
julia> x -> x^2 + 2x - 1
#1 (generic function with 1 method)

julia> function (x)
           x^2 + 2x - 1
       end
#3 (generic function with 1 method)
```

这样就创建了一个接受一个参数 x 并返回当前值的多项式 x^2+2x-1 的函数。

匿名函数最主要的用法是传递给接收函数作为参数的函数。一个经典的例子是 `map`，为数组的每个元素应用一次函数，然后返回一个包含结果值的新数组：

```
julia> map(round, [1.2, 3.5, 1.7])
3-element Vector{Float64}:
 1.0
 4.0
 2.0
```

3.3.5 匿名函数

匿名函数

如果做为第一个参数传递给 map 的转换函数已经存在，那直接使用函数名称是没问题的。但是通常要使用的函数还没有定义好，这样使用匿名函数就更加方便：

```
julia> map(x -> x^2 + 2x - 1, [1, 3, -1])  
3-element Vector{Int64}:  
 2  
14  
-2
```

接受多个参数的匿名函数写法可以使用语法 $(x,y,z) \rightarrow 2x+y-z$ ，而无参匿名函数写作 $() \rightarrow 3$ 。

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

3.3.7 函数示例-构建及调用

示例：计算A、B、C多个矩阵的乘积

$$A = \begin{vmatrix} 1 & 2 \\ 3 & 4 \end{vmatrix} \quad B = \begin{vmatrix} 5 & 6 \\ 7 & 8 \end{vmatrix} \quad C = \begin{vmatrix} 9 \\ 10 \end{vmatrix}$$

计算多个矩阵的乘积

```
function multiply_matrices_test(matrices::Vector{Array{Float64}})
    result = matrices[1]
    for i in 2:length(matrices)
        result = result * matrices[i]
    end
    return result
end
```

定义矩阵A、B、C

A = [1.0 2.0; 3.0 4.0]

B = [5.0 6.0; 7.0 8.0]

C = [9.0; 10.0]

matrices = [A, B, C]

result = multiply_matrices_test(matrices)

主函数与子函数在同一脚本

输出结果为：

2-element Vector{Float64}:
391.0
887.0

3.3.7 函数示例-构建及调用

示例：计算A、B、C多个矩阵的乘积 $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$ $C = \begin{bmatrix} 9 \\ 10 \end{bmatrix}$

计算多个矩阵的乘积

```
function multiply_matrices_test(matrices::Vector{Array{Float64,1}})
    result = matrices[1]
    for i in 2:length(matrices)
        result = result * matrices[i]
    end
    return result
end
```

```
include("multiply_matrices.jl")
```

```
A = [1.0 2.0; 3.0 4.0]
```

```
B = [5.0 6.0; 7.0 8.0]
```

```
C = [9.0; 10.0]
```

```
matrices = [A, B, C]
```

```
result = multiply_matrices_test(matrices)
```

实现函数脚本调用

主函数与子函数在不同脚本



形成子函数文件



构建主函数文件

3.3.7 函数示例-构建及调用

`include("函数文件路径")` 实现函数脚本调用

□ 资源管理器打开至函数文件所在目录



- 路径 ✓

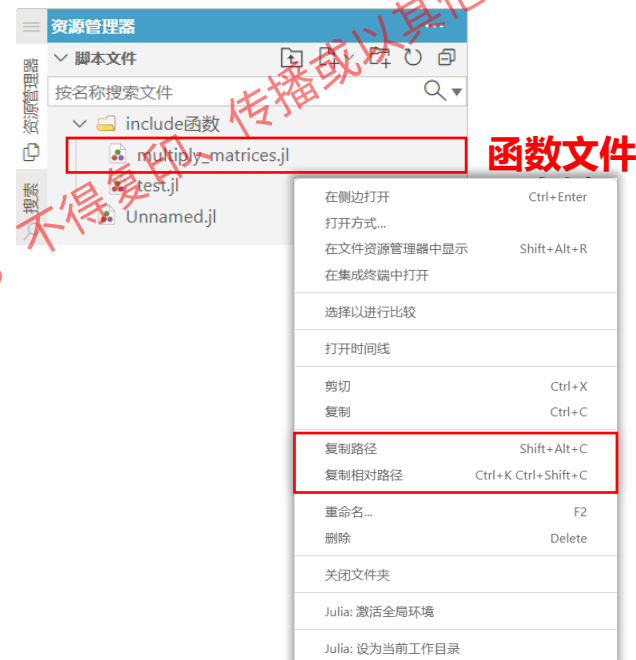
C:/Users/TR/Desktop/.../脚本文件/include函数/multiply_matrices.jl

- 相对路径 ✓

multiply_matrices.jl

提示: `pwd()` 命令也可显示当前文件夹的路径, 通过拼接构成绝对路径 `include(pwd()*"/multiply_matrices.jl")`

□ 资源管理器未打开至函数文件所在目录



- 路径 ✓

C:/Users/TR/Desktop/.../脚本文件/include函数/multiply_matrices.jl

- 相对路径 ✗

include函数/multiply_matrices.jl

3.3.7 函数示例-构建及调用

注意事项

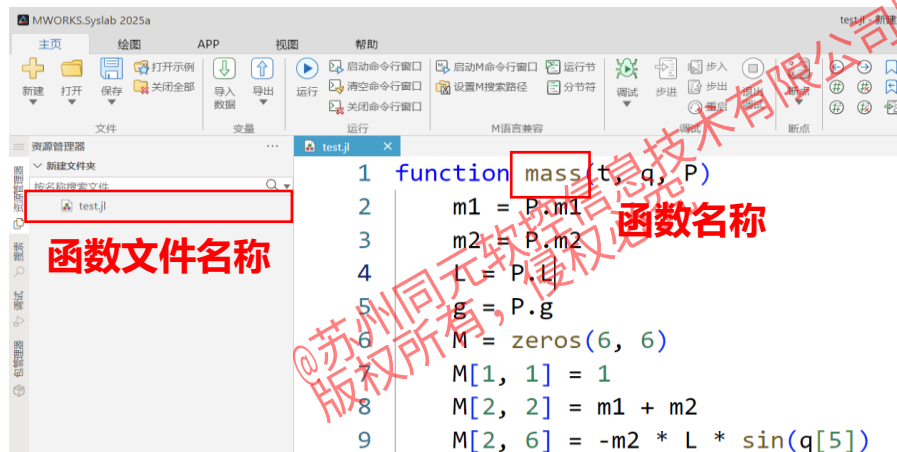
□ 关于函数文件名称

M软件:

- 函数文件名称必须与文件中定义的主函数名称完全一致
例如: 若函数定义为 `function y = CalSum(x)`, 则文件必须保存为 `CalSum.m`, 否则会报错

Syslab:

- 函数文件名称与文件中定义的函数名称无关联



Syslab

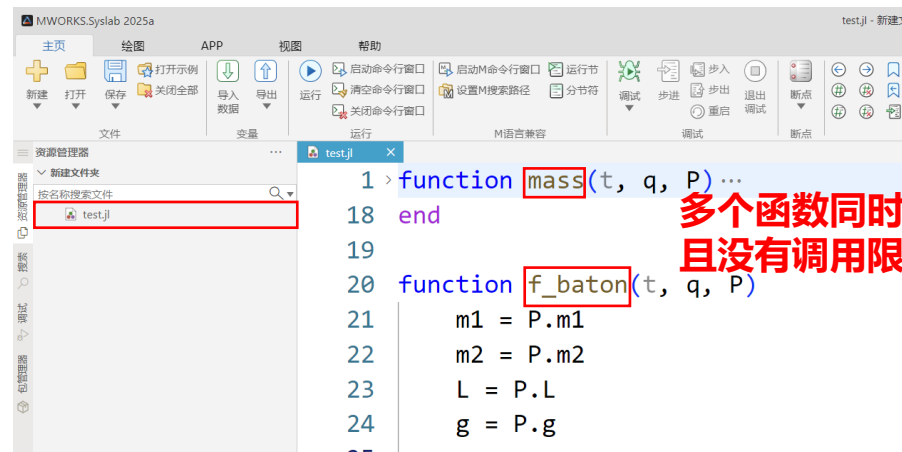
□ 关于函数文件中定义多个函数

M软件:

- 函数文件支持一个主函数+多个子函数的结构
- 每个函数文件只能有一个主函数, 其名称必须与文件名一致
- 在主函数之后, 可以定义多个子函数, 子函数仅在该文件内可见, 无法被外部直接调用

Syslab:

- 函数文件可定义多个函数, 没有主函数或子函数的严格限制



Syslab

3.4 代码调试

代码编写很难一次成功，往往需要进行代码调试，列举2个常见的错误情况

□ 情况1-代码运行报错

```
test.jl × multiply_matrices.jl
1 include("multiply_matrices.jl")
2 # 定义矩阵A、B、C
3 A = [1.0 2.0; 3.0 4.0]
4 B = [5.0 6.0; 7.0 8.0]
5 C = [9.0; 10.0] 逗号改为分号
6 matrices = [A; B; C]
7 result = multiply_matrices_test(matrices)
```

运行报错

```
调试控制台 终端
julia> 正在运行 test.jl
ERROR: ArgumentError: number of columns of each array must match (got (2, 2, 1))
Stacktrace:
 [1] _typed_vcat{unused#::Type{Float64}, A::Tuple{Matrix{Float64}, Matrix{Float64}, Vector{Float64}}}
   @ Base .\abstractarray.jl:1689
 [2] typed_vcat
   @ .\abstractarray.jl:1703 [inlined]
 [3] vcat{::Matrix{Float64}, ::Matrix{Float64}, ::Vector{Float64}}
   @ Base .\array.jl:1958
 [4] top-level scope
   @ C:\Users\TR\Desktop\include函数\test.jl:6
```

报错原因：列数不匹配

出错代码位置：第6行

□ 情况2-代码运行不报错，结果错误

```
test.jl × multiply_matrices.jl ×
1 # 计算多个矩阵的乘积
2 function multiply_matrices_test(matrices::Vector{Matrix{Float64}})
3     result = matrices[1]
4     for i in 1:length(matrices)
5         result = result * matrices[i]
6     end
7     return result
8 end
```

结果错误

```
调试控制台 终端
julia> 正在运行 test.jl
2-element Vector{Float64}:
 2165.0
 4721.0
```

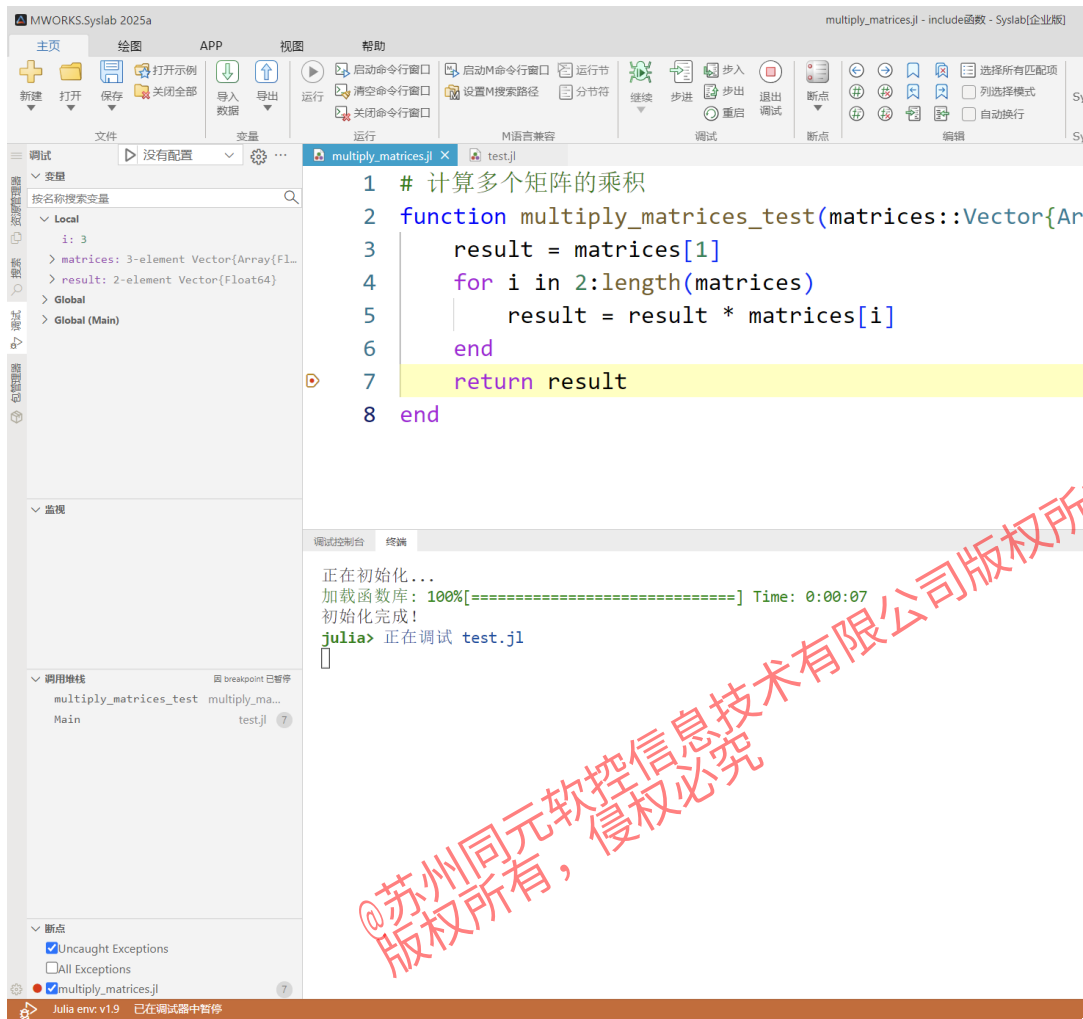
正确结果为：

2-element Vector{Float64}:
391.0
887.0

说明：在此例中，情况1无需调试，情况2需调试

3.4 代码调试

Syslab提供代码调试器，支持对代码文件的**单步调试**、**断点调试**、**添加监视**、**查看调用堆栈**等，并提供**交互式调试控制台**



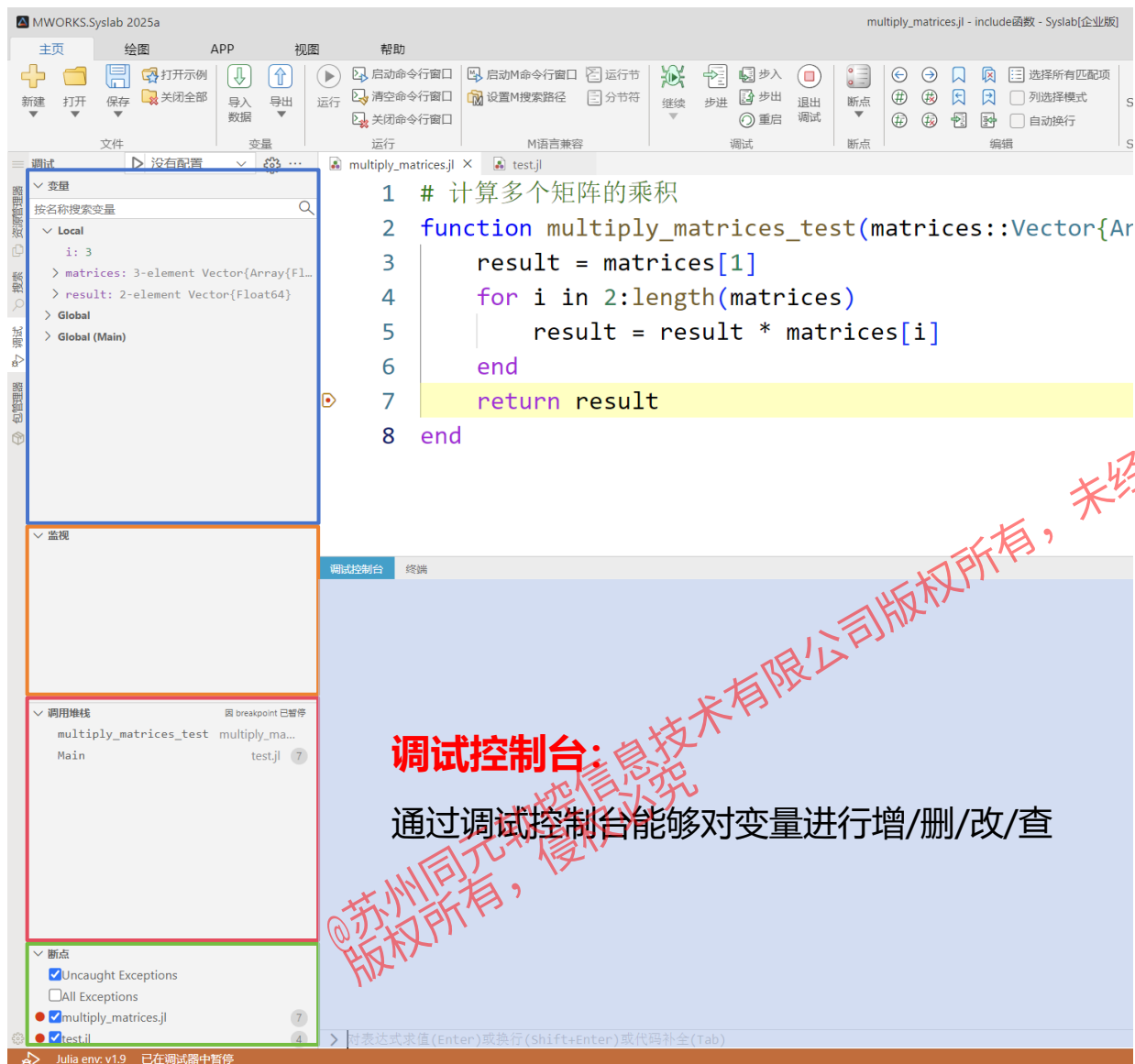
操作步骤:

1. 在脚本打上断点
2. 点击菜单栏中"启动调试"或F5进入DeBug模式

说明:

- **继续(F5)**: 继续运行调试
- **步进(F10)**: 单步执行遇到子函数时不会进入子函数内，而是将子函数整个执行完再停止
- **步入(F11)**: 单步执行遇到子函数就进入并且继续单步执行
- **步出(Shift+F11)**: 当单步执行到子函数内时，执行完子函数余下部分，并返回到上一层函数
- **重启(Ctrl+Shift+F5)**: 重新启动调试
- **退出调试(Shift+F5)**: 停止调试

3.4 代码调试



说明：

□ 变量：

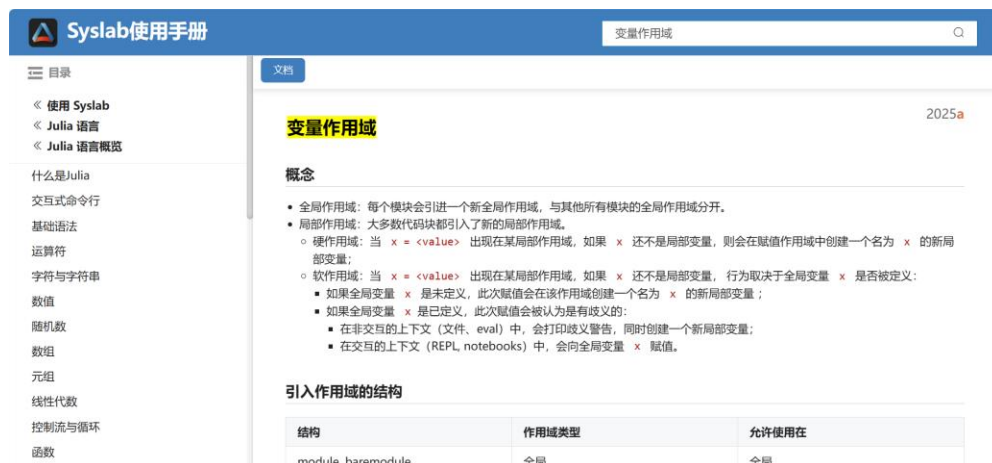
- Local：函数局部变量的值
- Global：函数全局变量的值，变量前具有global前缀
- Global(Main)：主程序全局变量的值

□ 监视：输入变量名，监视变量的变化

□ 调用堆栈：用于了解函数间的调用关系和逻辑

□ 断点：模型中所有断点的信息

关于变量作用域相关内容，可查看帮助文档



3.4 代码调试

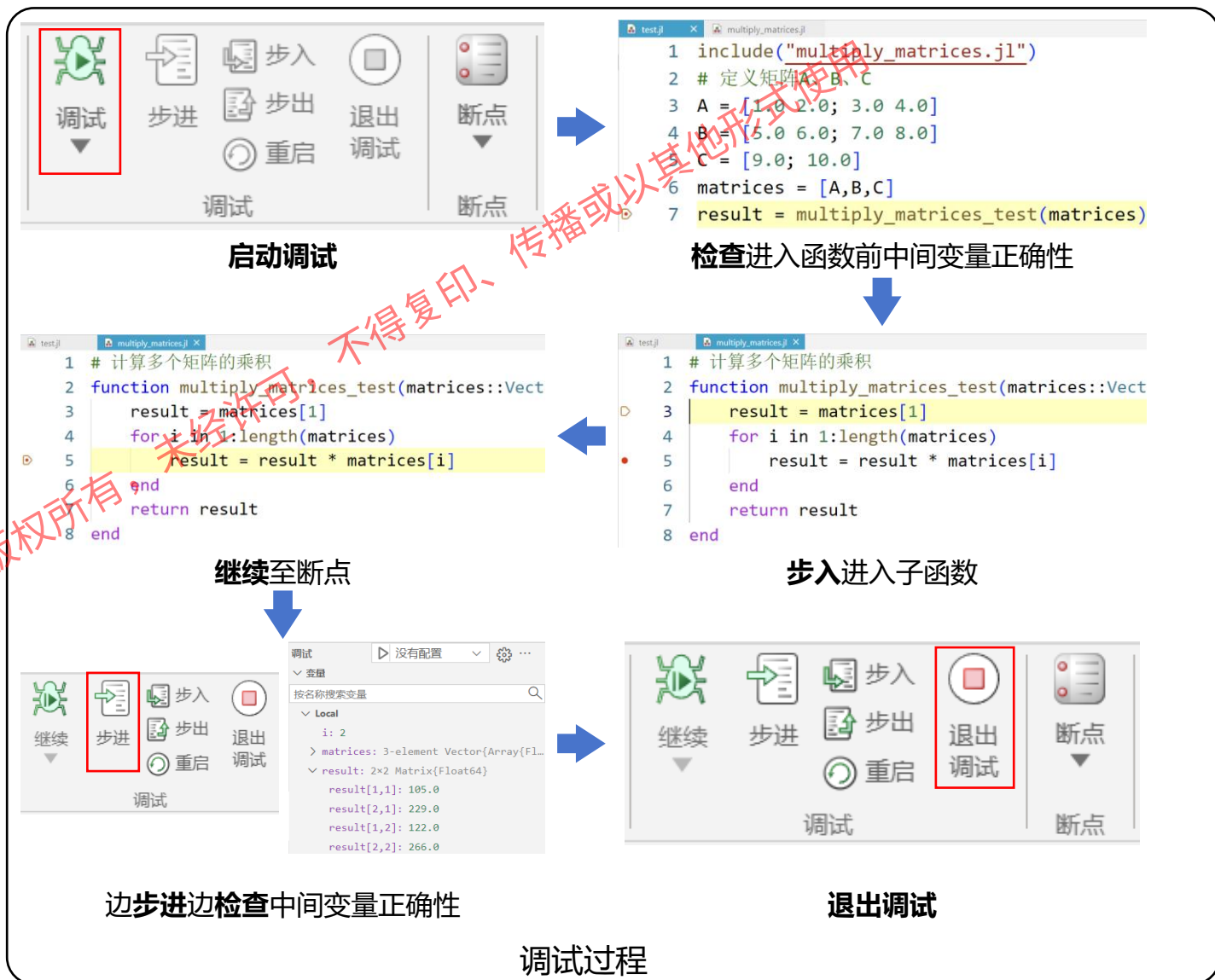
操作步骤:

1. 在脚本打上断点
2. 点击菜单栏中"启动调试"或F5进入DeBug模式

```
test.jl x multiply_matrices.jl
1 include("multiply_matrices.jl")
2 # 定义矩阵A、B、C
3 A = [1.0 2.0; 3.0 4.0]
4 B = [5.0 6.0; 7.0 8.0]
5 C = [9.0; 10.0]
断点1 matrices = [A,B,C]
• 7 result = multiply_matrices_test(matrices)
```

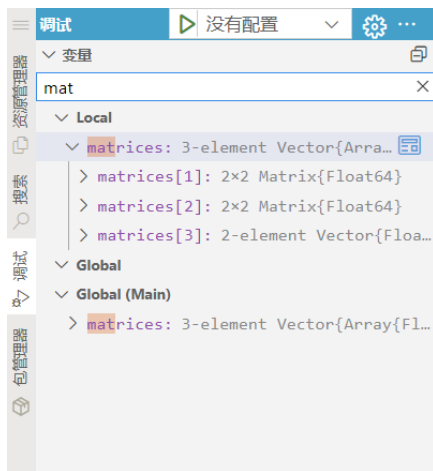
```
test.jl x multiply_matrices.jl x
1 # 计算多个矩阵的乘积
2 function multiply_matrices_test(matrices::
3     result = matrices[1]
断点2 for i in 1:length(matrices)
5     result = result * matrices[i]
6 end
7 return result
8 end
```

打断点

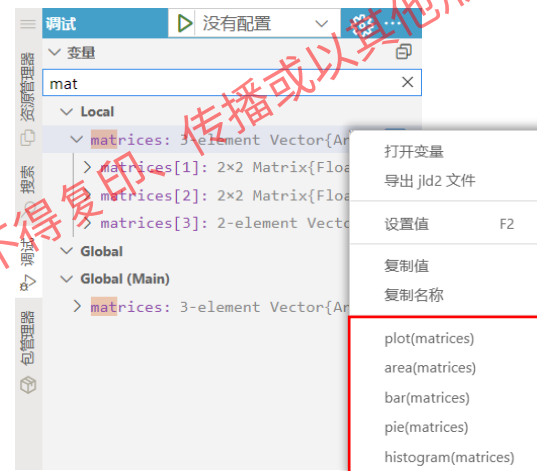


3.4 代码调试

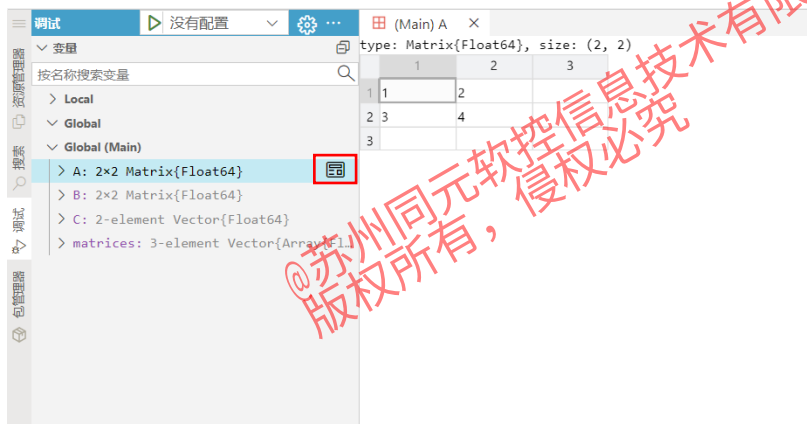
□ 支持变量搜索



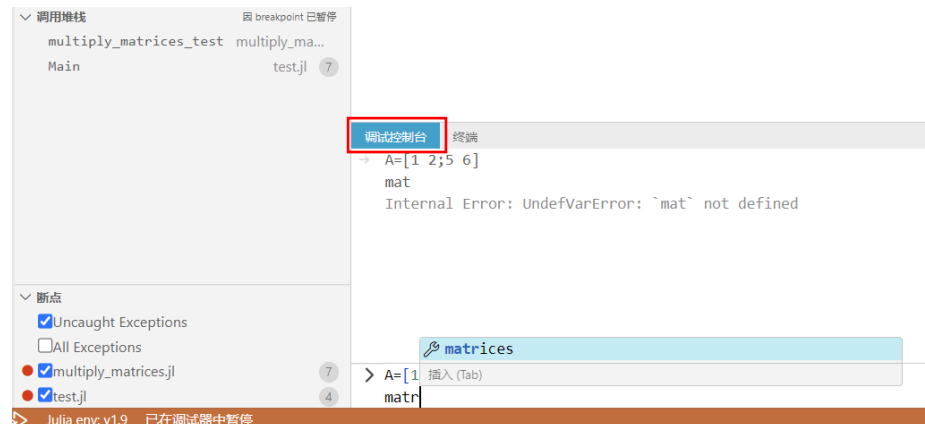
□ 支持变量右键直接绘图



□ 支持查看变量表格视图



□ 调试控制台支持换行输入、代码补全



3.4 代码调试-性能分析

支持程序计时及程序性能采样分析

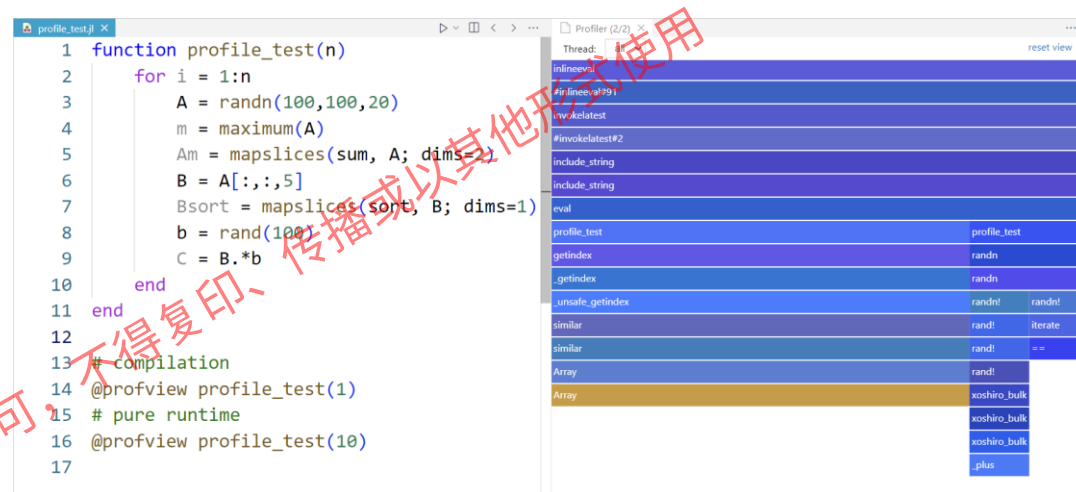
□ 提供多种宏调用方式，支持对表达式计时和查看内存分配情况

- `@time`对表达式计时和查看内存分配：运行时间；内存分配次数；总计分配的内存数；整理内存所用的时间
- `@elapsed`对表达式计时，以浮点数的形式返回执行所用的秒数。
- `@btime`提供在基准测试期间测量的最小运行时间。
- `@benchmark`用于对Julia表达式进行基准测试。其运行结果主要包括：运行次数、最小用时、最大用时、中位数用时、平均用时，以及占用内存等
- `@profview`对代码进行采样分析，并输出火焰图

```
time_test.jl
1 x = rand(10,10);
2
3 @time x * x; # 第一次存在编译时间
4
5 @time begin
6     x * x
7 end;
```

```
julia> 正在运行 time_test.jl
0.003447 seconds (57 allocations: 3.625 KiB, 96.75% compilation time)
0.000011 seconds (1 allocation: 896 bytes)
```

@time对表达式计时和查看内存分配



性能采样分析结果

Syslab使用手册

全局搜索

目录

- 使用 Syslab
- Julia 语言
- Julia 语言概览
- Julia 高性能编程**
 - 核心思路
 - 性能分析
 - 常见性能优化
 - 数组
 - for 循环优化
 - 类型稳定
 - 数值类型
 - 多重派发
 - 内存优化
 - 内置函数使用
 - 外部语言接口

Julia 高性能编程

2025

Julia 是一门灵活的动态语言，适用于科学计算和数值计算，并且性能可与传统的静态类型语言媲美。写出能用的 Julia 代码很容易，但写出高性能的 Julia 代码则需要长时间的训练，以及充足的知识储备。本节主要介绍 Julia 性能优化指南，帮助用户写出高性能代码。

核心思路

- 检查性能热点
- 写对编译器友好的代码
 - 确保类型稳定
 - 确保结构体或矩阵的元素类型是具体类型
- 避免不必要的内存操作
- 避免不必要的数组计算
- 使用高效的算法和数据结构
- 广播和向量化编程并不一定是好的
- 更好地利用硬件的计算能力
- 在单线程足够高效后再考虑多线程，因为否则可能会带来负优化

性能分析

帮助文档-Julia高性能编程经验

04

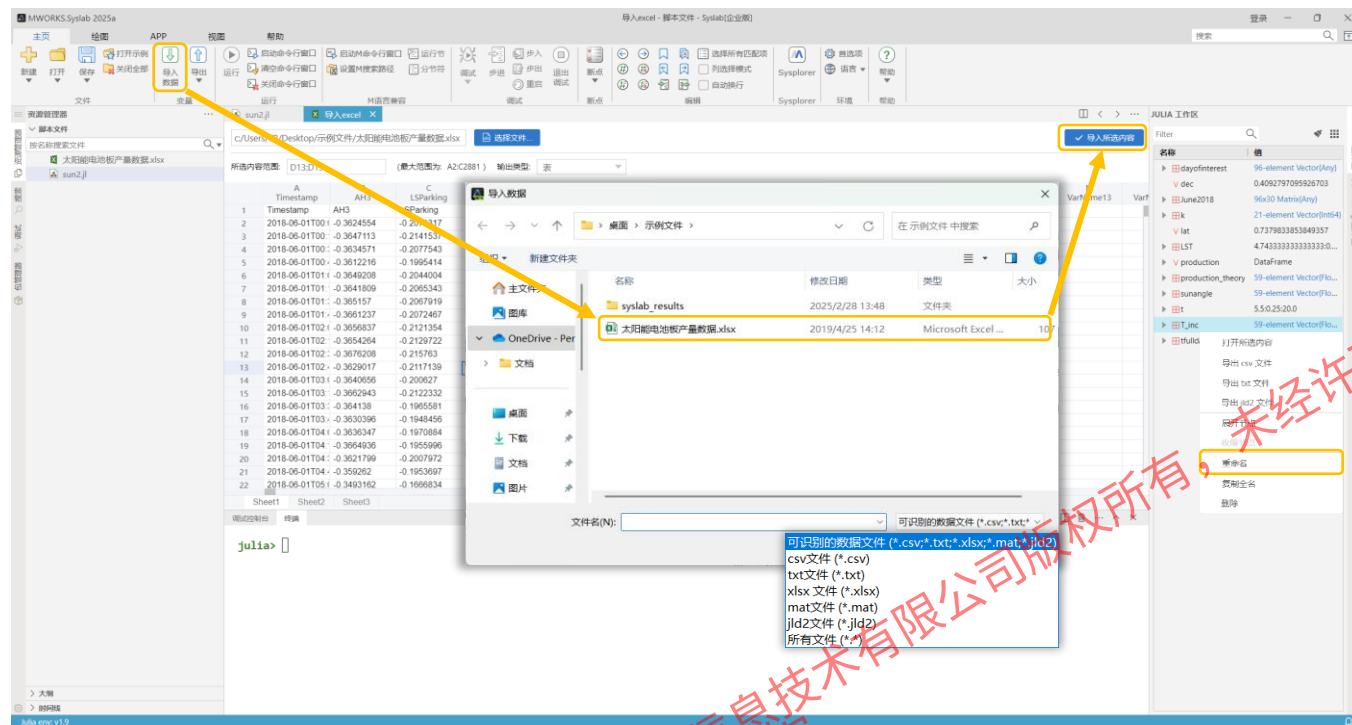
PART 04 ➔

数据导入导出及可视化

@苏州同元软件技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

4.1.1 数据导入

数据导入：可将数据文件直接导入至工作空间进行使用



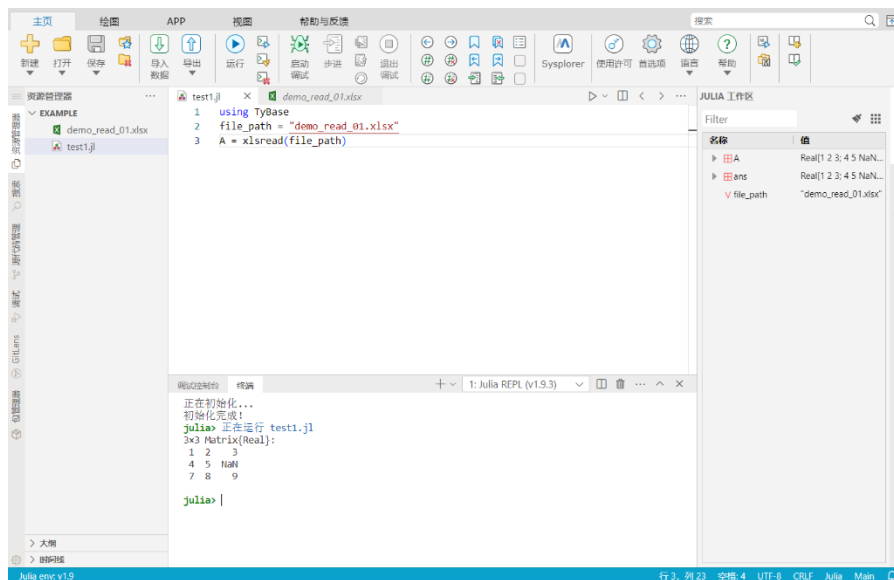
◆ 支持导入文件格式

- 文本: txt、csv
- Excel: xlsx
- Mat
- jld2

◆ 工作区查看变量，可对变量重命名

使用函数读取表格：xlsread

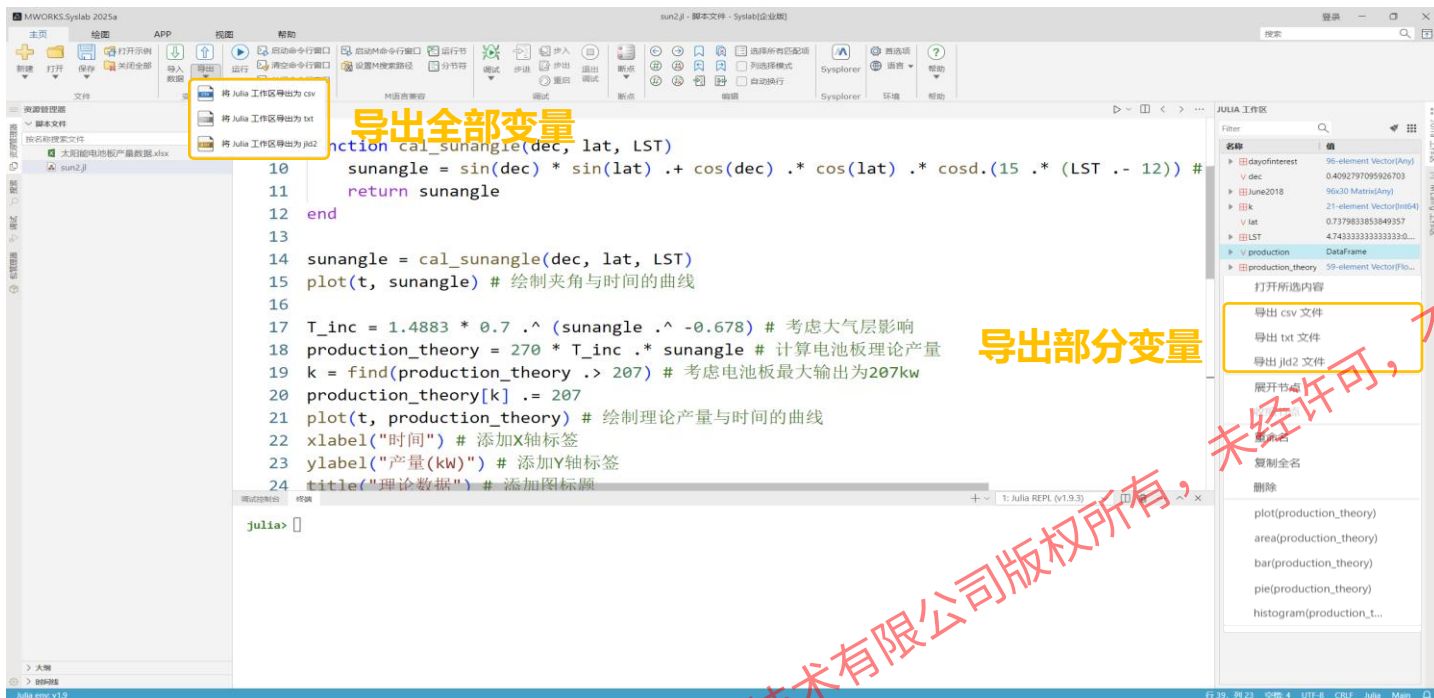
- 1.将需要读取的文件放置在 Syslab 打开的当前运行脚本的同级目录下。
- 2.设置file_path为当前文件名，既：file_path = "demo_read_01.xlsx"。
- 3.xlsread(file_path) 读取相对路径下工作表中的数值数据。



4.1.2 数据导出

结果导出：所有计算结果和部分计算结果可直接导出为csv、txt、jld2格式

使用函数将变量保存在mat/jld2/ASCII：save



说明：

◆ 支持导出文件格式

- csv
- txt
- jld2

◆ 支持导出全部和部分计算结果

```
using TyBase

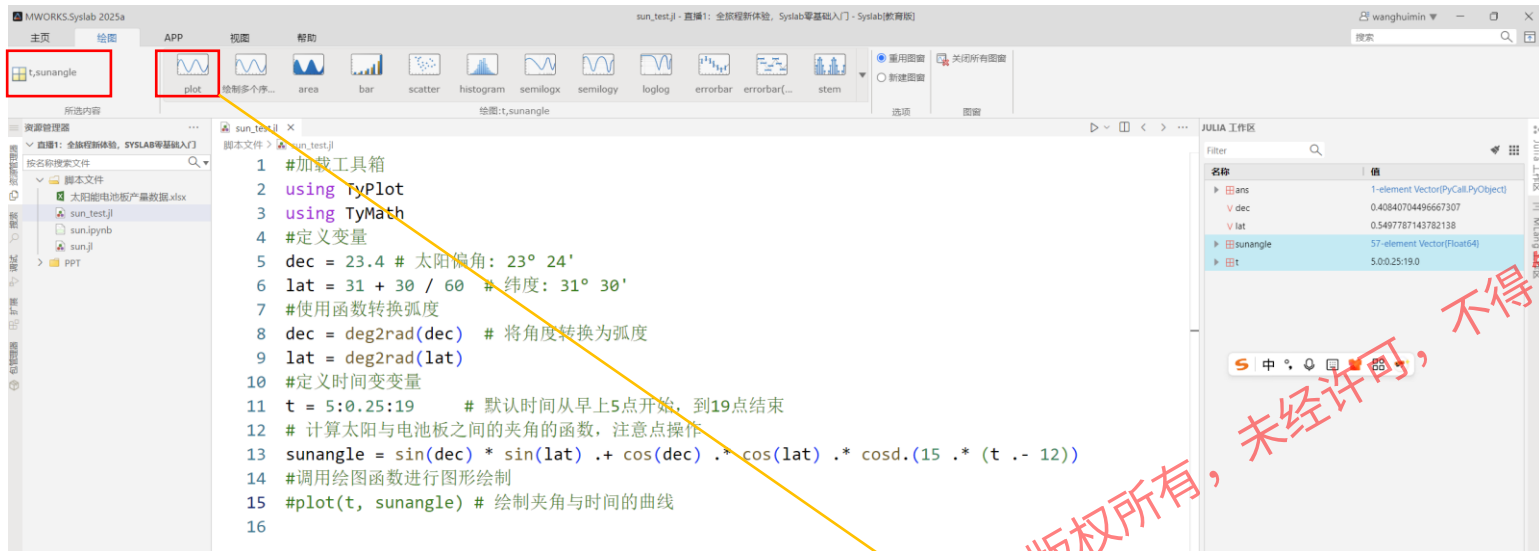
p = rand(1, 10)
q = ones(10)

filename = "test.mat"

save(filename, "-mat"; p, q)
```

4.2 数据可视化-图形化操作

绘图页中具备多种类型图形绘制功能，可直接选择变量进行绘制

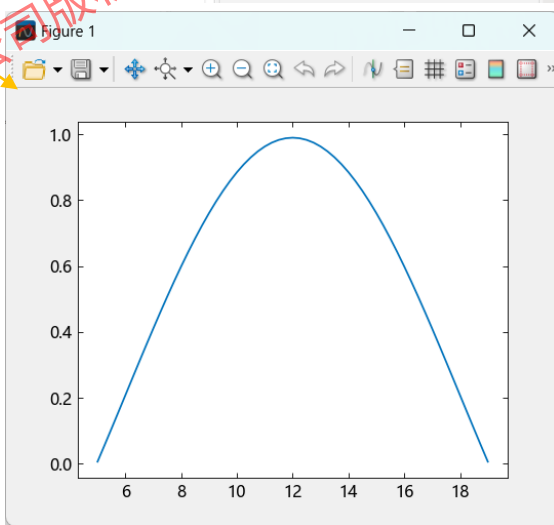


```
using TyPlot
using TyMath

dec = 23.4 # 太阳偏角: 23° 24'
lat = 31 + 30 / 60 # 纬度: 31° 30'
dec = deg2rad(dec) # 将角度转换为弧度
lat = deg2rad(lat)
t = 5:0.25:19 # 默认时间从早上5点开始, 到19点结束
# 计算太阳与电池板之间的夹角的函数
sunangle = sin(dec) * sin(lat) .+ cos(dec) .*
cos(lat) .* cosd.(15 .* (t .- 12))
```

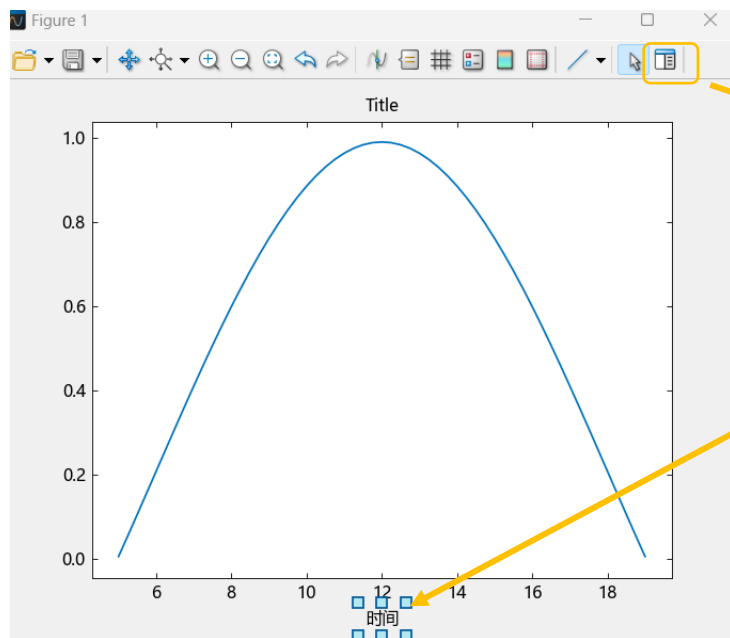
选择工作区变量，可视化绘图步骤：

1. 工作空间左键选择绘图的变量
2. 在“绘图”菜单栏选择图类型

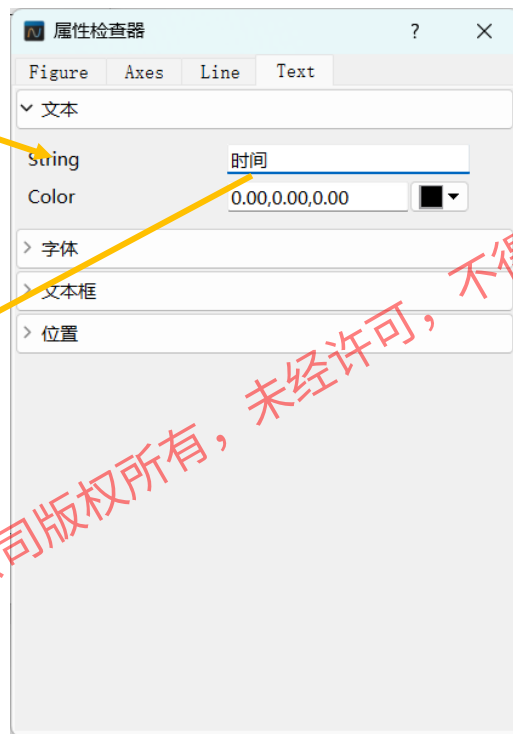


4.2 数据可视化-图形化操作

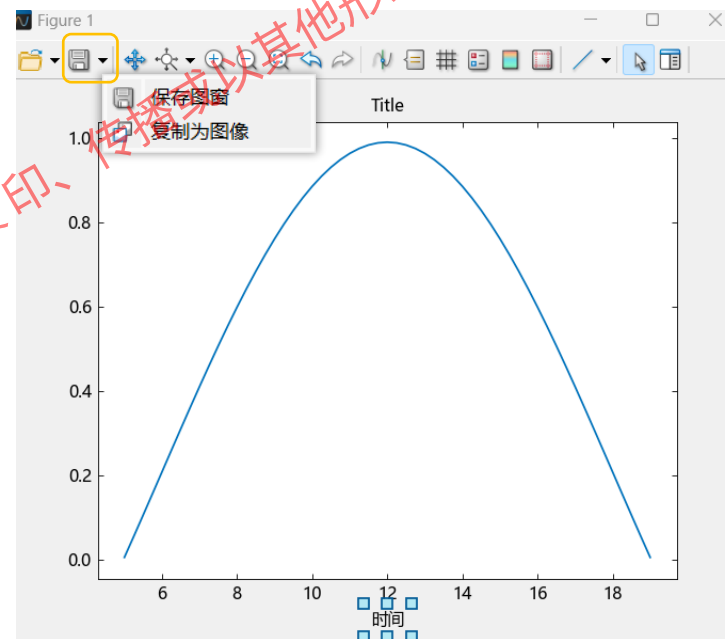
可在图形窗口直接配置属性，导出图片



属性检查器



导出图片

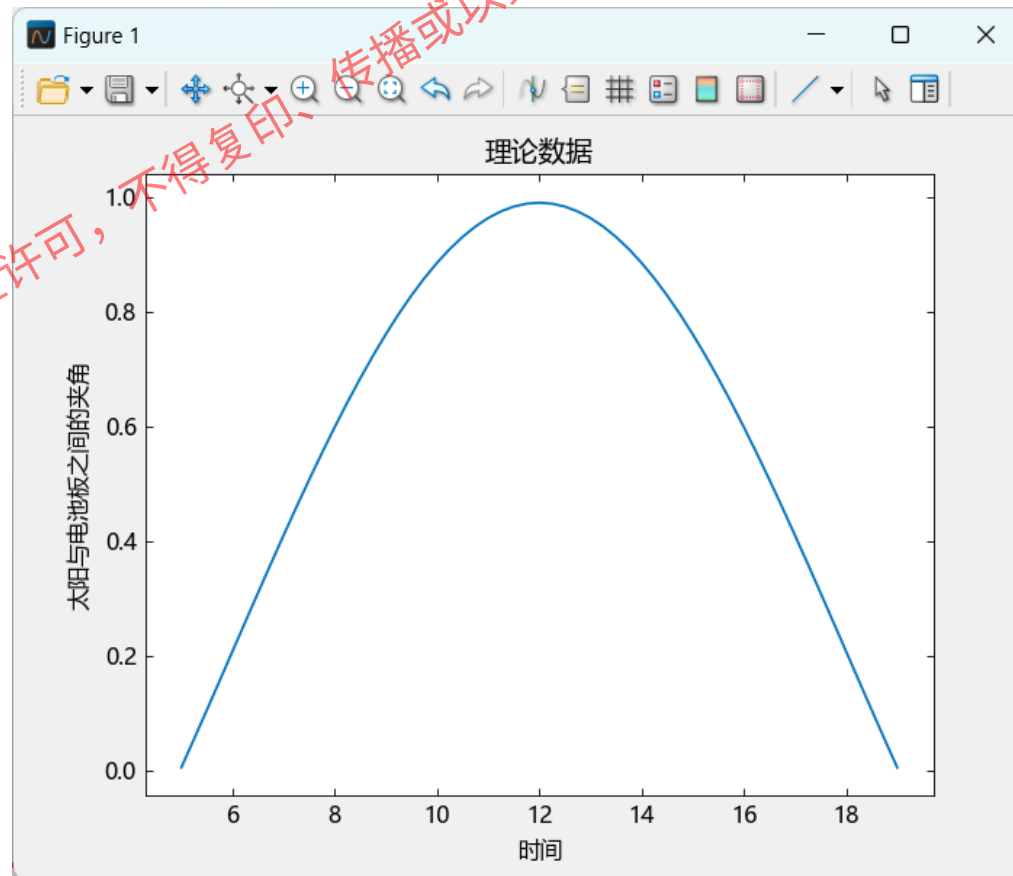


说明:

- 支持设置图窗颜色等属性
- 支持设置坐标轴字体、刻度、标尺、网格、标签、框样式、位置等属性
- 支持设置图线颜色、标记、图例等属性

4.2 数据可视化-函数操作

```
#加载工具箱
using TyPlot
using TyMath
#定义变量
dec = 23.4 # 太阳偏角: 23° 24'
lat = 31 + 30 / 60 # 纬度: 31° 30'
dec = deg2rad(dec) # 将角度转换为弧度
lat = deg2rad(lat)
t = 5:0.25:19 # 默认时间从早上5点开始, 到19点结束
# 计算太阳与电池板之间的夹角的函数
sunangle = sin(dec) * sin(lat) .+ cos(dec) .* cos(lat) .*
cosd.(15 .* (t .- 12))
plot(t, sunangle) # 绘制夹角与时间的曲线
xlabel("时间") # 添加x轴标签
ylabel("太阳与电池板之间的夹角") # 添加y轴标签
title("理论数据") # 添加图标题
```



05

PART 05 ➔

太阳能电池性能验证案例

@苏州同元软件技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

5.1 Syslab一般应用流程



Syslab
科学计算软件



理论分析



代码构建



运行调试



结果处理

需求分析、机理分析、
逻辑推导

函数构建、脚本构建
函数调用

Debug调试、代码运行

数据可视化、数据分析、
数据/文件IO

5.2 实例题目

题目：苏州某地安装有太阳能电池板，拟对比**实际产量**与**理论产量**，验证其性能

已知数据：

- ◆ 当地**太阳赤纬角**约为 $23^{\circ} 24'$ ，**纬度**为 $31^{\circ} 30'$
- ◆ **电池板面积**为 270 m^2 ，最大输出为 207 kw
- ◆ 电池板**某月份的产量数据**，提供Excel文件

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究



5.3 理论分析

题目：苏州某地安装有太阳能电池板，拟对比**实际产量**与**理论产量**，验证其性能

实际监测数据

理论计算数据

已知数据：

- ◆ 当地**太阳赤纬角**约为 $23^{\circ} 24'$ ，**纬度**为 $31^{\circ} 30'$
- ◆ **电池板面积**为 270 m^2 ，最大输出为 207 kw
- ◆ 电池板**某月份的产量数据**，提供Excel文件

理论公式：

- ◆ 太阳与电池板之间的夹角公式

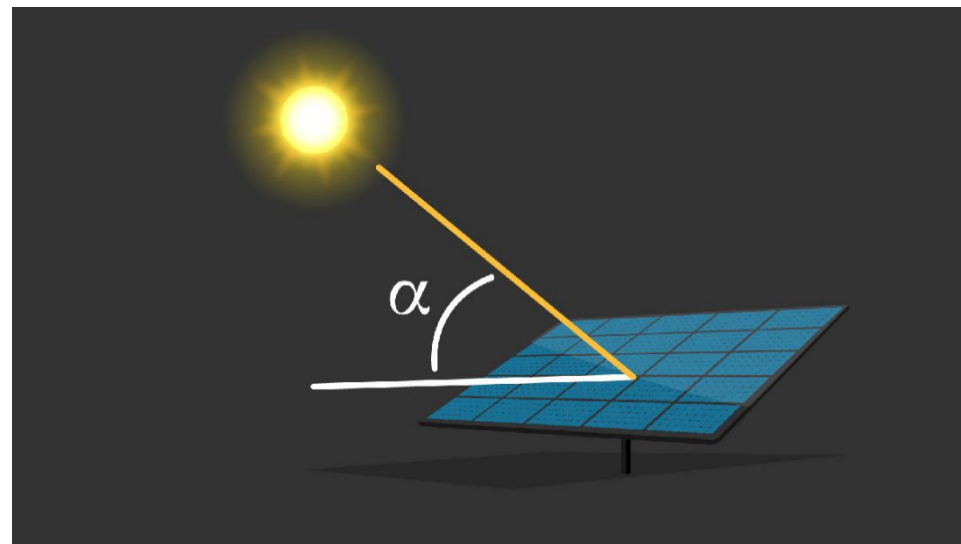
$$\sin(\alpha) = \sin(\underbrace{\delta}_{\text{太阳赤纬角}}) \cdot \sin(\underbrace{\varphi}_{\text{纬度}}) + \cos(\underbrace{\delta}_{\text{太阳赤纬角}}) \cdot \cos(\underbrace{\varphi}_{\text{纬度}}) \cdot \cos(15 \cdot \underbrace{t}_{\text{时间}} - 12)$$

- ◆ 考虑大气层影响的经验系数公式

$$T_{inc} = 1.4883 \times 0.7^{\sin(\alpha) - 0.678}$$

- ◆ 电池板产量公式

$$P = \underbrace{S}_{\text{电池板面积}} \cdot \sin(\alpha) \cdot T_{inc}$$



Thanks.

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入工业创新，共创先进软件